

[Página Principal](#) / [Mis cursos](#) / [AED-2024-IntLib-\(Feb2025\)](#) / [Ficha 14](#) / [Cuestionario 14 \[Temas: hasta Ficha 14\]](#)

Comenzado el	martes, 4 de marzo de 2025, 22:41
Estado	Finalizado
Finalizado en	miércoles, 5 de marzo de 2025, 11:48
Tiempo empleado	13 horas 7 minutos
Puntos	22/24
Calificación	9 de 10 (92%)

Pregunta **1**

Correcta

Se puntúa 2 sobre 2

El cálculo del valor a^n (para simplificar, asumimos $a > 0$ y $n \geq 0$ y sabiendo que si $n = 0$ entonces $a^0 = 1$) es igual a multiplicar n veces el número a por sí mismo. Por caso, $5^3 = 5 * 5 * 5 = 125$. Sabiendo esto, ¿cuál de las siguientes sería una *definición recursiva* matemáticamente correcta de la operación $potencia(a, n)$?

Seleccione una:

☐ a.

$$potencia(a, n) \begin{cases} = 1 & \text{si } n == 0 \\ = a * potencia(a, n) & \text{si } n > 0 \end{cases}$$

(a y n enteros, a > 0 y n >= 0)

☐ b.

$$potencia(a, n) \begin{cases} = 1 & \text{si } n == 0 \\ = a * potencia(a-1, n) & \text{si } n > 0 \end{cases}$$

(a y n enteros, a > 0 y n >= 0)

☒ c.

$$potencia(a, n) \begin{cases} = 1 & \text{si } n == 0 \\ = a * potencia(a, n-1) & \text{si } n > 0 \end{cases}$$

(a y n enteros, a > 0 y n >= 0)

✓ ¡Correcto!

☐ d.

$$potencia(a, n) \begin{cases} = 1 & \text{si } n == 0 \\ = a + potencia(a, n-1) & \text{si } n > 0 \end{cases}$$

(a y n enteros, a > 0 y n >= 0)

¡Correcto!

La respuesta correcta es:

$$potencia(a, n) \begin{cases} = 1 & \text{si } n == 0 \\ = a * potencia(a, n-1) & \text{si } n > 0 \end{cases}$$

(a y n enteros, a > 0 y n >= 0)

Pregunta 2

Correcta

Se puntúa 1 sobre 1

Suponga que se quiere plantear una **definición recursiva** del concepto de **bosque**. ¿Cuál de las siguientes propuestas generales es correcta y constituye la mejor definición?

Seleccione una:

- ☒ a. Un bosque es un conjunto de árboles que puede estar vacío, o puede contener uno o más árboles agrupados con otro bosque. ✓ ¡Correcto!
- ☐ b. Un bosque es un bosque.
- ☐ c. Un bosque es un conjunto de árboles que puede estar vacío, o puede contener n árboles (con $n > 0$).
- ☐ d. Un bosque es un conjunto que puede contener uno o más árboles agrupados con otro bosque.

¡Correcto!

La respuesta correcta es:

Un bosque es un conjunto de árboles que puede estar vacío, o puede contener uno o más árboles agrupados con otro bosque.

Pregunta 3

Correcta

Se puntúa 1 sobre 1

¿Cuáles de los siguientes son **factores esenciales** a tener en cuenta cuando se realiza el *análisis de la eficiencia de un algoritmo* (por ejemplo, para efectuar una comparación de rendimiento entre dos algoritmos que resuelven el mismo problema)?

Seleccione una o más de una:

- ☒ a. El consumo de memoria. ✓ ¡Correcto!
- ☐ b. El diseño de la interfaz de usuario.
- ☒ c. La complejidad aparente del código fuente. ✓ ¡Correcto!
- ☒ d. El tiempo de ejecución. ✓ ¡Correcto!

¡Correcto!

Las respuestas correctas son:

El tiempo de ejecución.,

El consumo de memoria.,

La complejidad aparente del código fuente.

Pregunta 4

Correcta

Se puntúa 1 sobre 1

Suponga que la versión recursiva de la función para el cálculo del factorial que se presentó en clases fuese redefinida en la forma siguiente:

```
def factorial(n):  
    f = n * factorial(n - 1)  
    if n == 0:  
        return 1  
    return f
```

¿Cuál es el problema (si lo hubiese) con esta versión de la función?

Seleccione una:

- ☒ a. Contiene todos los elementos que debería tener una función recursiva bien planteada, pero en el orden incorrecto: la condición para chequear si n es cero debería estar antes que la llamada recursiva para evitar la recursión infinita. ✓ ¡Correcto!
- ☐ b. No hay ningún problema. Funciona correctamente.
- ☐ c. Para calcular el factorial necesita un ciclo for que genere los números a multiplicar. Así planteada, la función hace un único primer cálculo incompleto y lo retorna.
- ☐ d. La función siempre retorna un 1, sin importar cual sea el valor de n.

¡Correcto!

La respuesta correcta es:

Contiene todos los elementos que debería tener una función recursiva bien planteada, pero en el orden incorrecto: la condición para chequear si n es cero debería estar antes que la llamada recursiva para evitar la recursión infinita.

Pregunta 5

Correcta

Se puntúa 2 sobre 2

A lo largo del curso hemos visto la forma de plantear programas controlados por menú de opciones, y sabemos que esos programas incluyen un ciclo para el control del proceso. Pero también podríamos hacer un planteo basado en recursión, sin usar el ciclo, en forma similar a la que se muestra aquí:

```
__author__ = 'Cátedra de AED'

def menu():
    print('1. Opcion 1')
    print('2. Opcion 2')
    print('3. Salir')
    op = int(input('Ingrese opcion: '))
    if op == 1:
        print('Eligio la opcion 1...')
    elif op == 2:
        print('Eligio la opcion 2...')
    elif op == 3:
        return
    menu()

# script principal...
menu()
```

¿Hay algún problema o contraindicación respecto del uso y aplicación de esta versión recursiva para la gestión de un menú? Seleccione la respuesta que mejor describa este punto.

Seleccione una:

- ☐ a. La versión recursiva que se mostró no funciona. Sólo permite cargar una vez la opción elegida, e inmediatamente se detiene.
 - ☐ b. Esta versión recursiva es completamente equivalente a la versión iterativa. No hay ningún motivo para preferir la una sobre la otra. Da lo mismo si el programador usa la recursiva o la iterativa.
 - ☐ c. No hay ningún problema ni contraindicación. Y no sólo eso: la versión recursiva es más simple y compacta que la versión basada en un ciclo, por lo cual es incluso preferible usarla.
 - ☒ d. Esta versión recursiva posiblemente sea más simple y compacta que la versión basada en un ciclo, pero la versión recursiva pide memoria de stack cada vez que se invoca, por lo que es más conveniente la iterativa. ✓
- ¡Correcto! De todos modos, si la idea es usar el menú sólo para un par de operaciones y terminar de inmediato, no habrá problema alguno. Lo malo sería que el programa funcione en forma permanente, las 24 horas del día y todos los días... pues en ese caso, se corre el serio riesgo de provocar un overflow de stack.

¡Correcto!

La respuesta correcta es:

Esta versión recursiva posiblemente sea más simple y compacta que la versión basada en un ciclo, pero la versión recursiva pide memoria de stack cada vez que se invoca, por lo que es más conveniente la iterativa.

Pregunta 6

Correcta

Se puntúa 2 sobre 2

El producto de dos números a y b mayores o iguales cero, es en última instancia *una suma*: se puede ver como sumar b veces el número a , o como sumar a veces el número b . Por ejemplo: $5 * 3 = 5 + 5 + 5$ o bien $5 * 3 = 3 + 3 + 3 + 3 + 3$. Sabiendo esto, se puede intentar hacer una definición recursiva de la operación `producto(a, b)` [$a \geq 0, b \geq 0$], que podría ser la que sigue:

$$\text{producto}(a, b) = \begin{cases} 0 & \text{si } a == 0 \text{ o } b == 0 \\ a + \text{producto}(a, b-1) & \text{si } a > 0 \text{ y } b > 0 \end{cases}$$

[a y b enteros, a >= 0 y b >= 0]

¿Es correcto el siguiente planteo de la función `producto(a, b)`?

```
def producto(a, b):  
    if a == 0 or b == 0:  
        return 0  
    return a + producto(a, b-1)
```

Seleccione una:

- ☐ a. No. La función sugerida siempre retornará 0, sean cuales fuesen los valores de a y b .
- ☐ b. No. La propia definición previa del concepto de producto es incorrecta: un producto es una multiplicación, y no una suma...
- ☐ c. No. La última línea debería decir `return a + producto(a-1, b-1)` en lugar de `return a + producto(a, b-1)`.
- ☒ d. Sí. Es correcto. ✓ ¡Correcto!

¡Correcto!

La respuesta correcta es:

Sí. Es correcto.

Pregunta 7

Correcta

Se puntúa 2 sobre 2

En la tabla siguiente se muestra un resumen comparativo entre las versiones iterativa (basada en ciclos) y recursiva del algoritmo de cálculo del término n -ésimo de la Sucesión de [Fibonacci](#). Note que se han dejado sin completar los casilleros que corresponden al tiempo de ejecución para ambos casos.

Cálculo del término n -ésimo de Fibonacci [F(n)]	Complejidad código fuente	Consumo de memoria	Tiempo de ejecución
Versión iterativa	Aceptable	Constante (no depende de n)	???
Versión recursiva	Optima	Proporcional a n (lineal)	???

¿Cuál es la estimación para el tiempo de ejecución de ambos algoritmos?

Seleccione una:

- ☐ a. Versión iterativa: tiempo proporcional a n (lineal) - Versión recursiva: tiempo proporcional a n (lineal)
- ☒ b. Versión iterativa: tiempo proporcional a n (lineal) ✓ ¡Correcto! Efectivamente... la cantidad de instancias recursivas que llegan a crearse a lo largo de todo el proceso recursivo está en el orden de 1.8^n (lo cual es muy mala noticia...)
- Versión recursiva: tiempo proporcional a b^n (exponencial, para algún $b > 1$)
- ☐ c. Versión iterativa: tiempo constante (no depende de n) - Versión recursiva: tiempo proporcional a b^n (exponencial, para algún $b > 1$)
- ☐ d. Versión iterativa: tiempo proporcional a b^n (exponencial, para algún $b > 1$) - Versión recursiva: tiempo proporcional a n (lineal)

¡Correcto!

La respuesta correcta es:

Versión iterativa: tiempo proporcional a n (lineal) - Versión recursiva: tiempo proporcional a b^n (exponencial, para algún $b > 1$)

Pregunta 8

Parcialmente correcta

Se puntúa 1 sobre 1

¿Cuáles de las siguientes propuestas generales son **ciertas** en relación al segmento de memoria conocido como **Stack Segment**?

Seleccione una o más de una:

- ☒ a. El Stack Segment se utiliza como soporte interno en el proceso de invocación a funciones (con o sin recursividad); ✓ ¡Correcto! se va llenando a medida que se desarrolla la cascada de invocaciones, y se vacía a medida que se produce el proceso de regreso o vuelta atrás.
- ☐ b. El Stack Segment se utiliza como soporte interno **solamente** en el proceso de invocación a funciones recursivas: se va llenando a medida que la cascada recursiva se va desarrollando, y se vacía a medida que se produce el proceso de regreso o vuelta atrás.
- ☐ c. El Stack Segment funciona como una cola (o fila) de datos (modalidad **FIFO**: First In - First Out): el primer dato en llegar, se almacena al frente del stack, y será por eso el primero en ser retirado.
- ☐ d. El Stack Segment funciona como una pila (o apilamiento) de datos (modalidad **LIFO**: Last In - First Out): el último dato en llegar, se almacena en la cima del stack, y será por eso el primero en ser retirado.

Ha seleccionado correctamente 1.

Las respuestas correctas son:

El Stack Segment funciona como una pila (o apilamiento) de datos (modalidad **LIFO**: Last In - First Out): el último dato en llegar, se almacena en la cima del stack, y será por eso el primero en ser retirado.,

El Stack Segment se utiliza como soporte interno en el proceso de invocación a funciones (con o sin recursividad): se va llenando a medida que se desarrolla la cascada de invocaciones, y se vacía a medida que se produce el proceso de regreso o vuelta atrás.

Pregunta 9

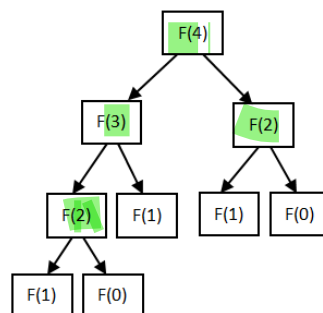
Correcta

Se puntúa 2 sobre 2

Sabemos que la recursión es una de las técnicas o estrategias básicas para el planteo de algoritmos y que una de sus ventajas es que permite el diseño de algoritmos compactos y muy claros, aunque al costo de usar algo de memoria extra en el stack segment y por consiguiente también un algo de tiempo extra por la gestión del stack. Algunos problemas pueden plantearse en forma directa procesando recursivamente diversos subproblemas menores. Tal es el caso de la *sucesión de Fibonacci*, en la cual cada término se calcula como la suma de los dos inmediatamente anteriores [esto se expresa con la siguiente relación de recurrencia: $F(n) = F(n-1) + F(n-2)$ con $F(0) = 0$ y $F(1) = 1$]. La función sencilla que mostramos a continuación que calcula el término n-ésimo de la sucesión en forma recursiva:

```
def fibo(n):  
    if n <= 1:  
        return 1  
    return fibo(n-1) + fibo(n-2)
```

Sin embargo, un inconveniente adicional es que la *aplicación directa* de *dos o más invocaciones recursivas* en el planteo de un algoritmo podría hacer que un mismo subproblema se resuelva más de una vez, incrementando el tiempo total de ejecución. Por ejemplo, para calcular $\text{fibo}(4)$ el árbol de llamadas recursivas es el siguiente:



y puede verse que en este caso, se calcula **2 veces** el valor de **fibo(2)**, **3 veces** el valor de **fibo(1)** y **2 veces** el valor de **fibo(0)**... con lo que la cantidad de llamadas a funciones para hacer *más de una vez el mismo trabajo* es de **7** ($= 2 + 3 + 2$).

Suponga que se quiere calcular el valor del sexto término de la sucesión. **Analice el árbol de llamadas recursivas que se genera al hacer la invocación $t = \text{fibo}(6)$. ¿Cuántas veces en total la función **fibo()** se invoca para hacer más de una vez el mismo trabajo, SIN incluir en ese conteo a las invocaciones para obtener **fibo(0)** y **fibo(1)**?**

Seleccione una:

- ☒ a. 10 ✓ ¡Correcto! Efectivamente: **fibo(4)** se invoca 2 veces, **fibo(3)** se invoca 3 veces y **fibo(2)** se invoca 5 veces...
- ☐ b. 0
- ☐ c. 11
- ☐ d. 12

¡Correcto!

La respuesta correcta es: 10

Pregunta 10

Correcta

Se puntúa 1 sobre 1

¿En cuáles de las siguientes situaciones el uso de *recursividad* está efectivamente recomendado? (Más de una respuesta puede ser válida. Marque todas las que considere correctas).

Seleccione una o más de una:

- ☐ a. Nunca.
- ☒ b. Recorrido y procesamiento de estructuras de datos no lineales como árboles y grafos. ✓ ¡Correcto!
- ☒ c. Generación y procesamiento de imágenes y gráficos fractales (figuras compuestas por versiones más simples de la misma figura original). ✓ ¡Correcto!
- ☐ d. Siempre que se pueda escribir una definición recursiva del problema.

¡Correcto!

Las respuestas correctas son:

Recorrido y procesamiento de estructuras de datos no lineales como árboles y grafos.,

Generación y procesamiento de imágenes y gráficos fractales (figuras compuestas por versiones más simples de la misma figura original).

Pregunta 11

Parcialmente correcta

Se puntúa 1 sobre 1

¿Cuáles de las siguientes son CIERTAS en relación al sistema de coordenadas de pantalla?

Seleccione una o más de una:

- ☒ a. Las filas de pantalla o de ventana con número de orden más alto, se encuentran más cerca del borde superior que las filas con número de orden más bajo. ✗ Incorrecto... en la pantalla las coordenadas pueden engañar...
- ☒ b. El origen del sistema de coordenadas se encuentra en el punto superior izquierdo de la pantalla o de la ventana que se esté usando. ✓ ¡Correcto!
- ☐ c. Las filas de pantalla o de ventana con número de orden más bajo, se encuentran más cerca del borde superior que las filas con número de orden más alto.
- ☐ d. El origen del sistema de coordenadas se encuentra en el punto inferior izquierdo de la pantalla o de la ventana que se esté usando.

Las respuestas correctas son:

El origen del sistema de coordenadas se encuentra en el punto superior izquierdo de la pantalla o de la ventana que se esté usando.,

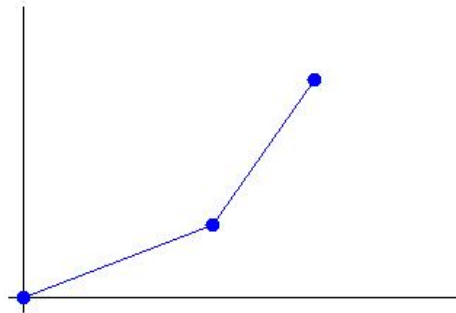
Las filas de pantalla o de ventana con número de orden más bajo, se encuentran más cerca del borde superior que las filas con número de orden más alto.

Pregunta 12

Correcta

Se puntúa 1 sobre 1

Suponga que se desea generar en Python el esquema de la **gráfica de una función**, como la que se muestra aquí (con las mismas proporciones y colores, aunque sin importar las coordenadas de visualización).



El siguiente programa está pensado para generar esa gráfica, pero dejando la función **function(canvas)** sin hacer:

```
__author__ = 'Cátedra de AED'

from tkinter import *

def function(canvas):
    pass

def render():
    # configuracion inicial de la ventana principal...
    root = Tk()
    root.title('Cuestionario')

    # calculo de resolucion en pixels de la pantalla...
    maxw = root.winfo_screenwidth()
    maxh = root.winfo_screenheight()

    # ajuste de las dimensiones y coordenadas de arranque de la ventana...
    root.geometry("%dx%d+%d+%d" % (maxw, maxh, 0, 0))

    # un lienzo de dibujo dentro de la ventana...
    canvas = Canvas(root, bg='white', width=maxw, height=maxh)
    canvas.grid(column=0, row=0)

    # desarrollar la gráfica...
    function(canvas)

    # lanzar el ciclo principal de control de eventos de la ventana...
    root.mainloop()

if __name__ == '__main__':
    render()
```

¿Cuál de las siguientes versiones de la función **function(canvas)** permitiría dibujar la gráfica sugerida?

Seleccione una:

☐ a.

```
def function(canvas):  
    # los ejes...  
    canvas.create_line((90, 400, 400, 400), fill='black')  
    canvas.create_line((100, 410, 100, 200), fill='black')  
  
    # las curvas...  
    canvas.create_line((100, 400, 230, 350), fill='black', width=3, dash=(5, 4))  
    canvas.create_line((230, 350, 300, 250), fill='black', width=3, dash=(5, 4))  
  
    # los puntos...  
    canvas.create_oval((95, 395, 105, 405), outline='blue', fill='blue')  
    canvas.create_oval((225, 345, 235, 355), outline='blue', fill='blue')  
    canvas.create_oval((295, 245, 305, 255), outline='blue', fill='blue')
```

☐ b.

```
def function(canvas):  
    # los ejes...  
    canvas.create_line((90, 400, 400, 400), fill='orange')  
    canvas.create_line((100, 410, 100, 200), fill='orange')  
  
    # las curvas...  
    canvas.create_line((100, 400, 230, 350), fill='black')  
    canvas.create_line((230, 350, 300, 250), fill='black')  
  
    # los puntos...  
    canvas.create_oval((95, 395, 105, 405), outline='blue', fill='blue')  
    canvas.create_oval((225, 345, 235, 355), outline='blue', fill='blue')  
    canvas.create_oval((295, 245, 305, 255), outline='blue', fill='blue')
```

☐ c.

```
def function(canvas):  
    # los ejes...  
    canvas.create_line((90, 400, 400, 400), fill='black')  
    canvas.create_line((100, 410, 100, 200), fill='black')  
  
    # las curvas...  
    canvas.create_line((100, 400, 230, 350), fill='black')  
    canvas.create_line((230, 350, 300, 250), fill='black')  
  
    # los puntos...  
    canvas.create_oval((95, 395, 105, 405), outline='blue', fill='red')  
    canvas.create_oval((225, 345, 235, 355), outline='blue', fill='red')  
    canvas.create_oval((295, 245, 305, 255), outline='blue', fill='red')
```

☒ d.

```
def function(canvas):  
    # los ejes...  
    canvas.create_line((90, 400, 400, 400), fill='black')  
    canvas.create_line((100, 410, 100, 200), fill='black')  
  
    # las curvas...  
    canvas.create_line((100, 400, 230, 350), fill='blue')  
    canvas.create_line((230, 350, 300, 250), fill='blue')  
  
    # los puntos...  
    canvas.create_oval((95, 395, 105, 405), outline='blue', fill='blue')  
    canvas.create_oval((225, 345, 235, 355), outline='blue', fill='blue')  
    canvas.create_oval((295, 245, 305, 255), outline='blue', fill='blue')
```

✓ ¡Correcto!

¡Correcto!

La respuesta correcta es:

```
def function(canvas):  
    # los ejes...  
    canvas.create_line((90, 400, 400, 400), fill='black')  
    canvas.create_line((100, 410, 100, 200), fill='black')  
  
    # las curvas...  
    canvas.create_line((100, 400, 230, 350), fill='blue')  
    canvas.create_line((230, 350, 300, 250), fill='blue')  
  
    # los puntos...  
    canvas.create_oval((95, 395, 105, 405), outline='blue', fill='blue')  
    canvas.create_oval((225, 345, 235, 355), outline='blue', fill='blue')  
    canvas.create_oval((295, 245, 305, 255), outline='blue', fill='blue')
```

Pregunta 13

Correcta

Se puntúa 1 sobre 1

Suponga que se desea generar Python *el dibujo de un botón para una aplicación que simule una interfaz visual de usuario*, como el que se muestra aquí (con las mismas proporciones y colores, aunque sin importar las coordenadas de visualización).



El siguiente programa está pensado para generar esa gráfica, pero dejando la función **button(canvas)** sin hacer:

```
__author__ = 'Cátedra de AED'

from tkinter import *

def button(canvas):
    pass

def render():
    # configuracion inicial de la ventana principal...
    root = Tk()
    root.title('Cuestionario')

    # calculo de resolucion en pixels de la pantalla...
    maxw = root.winfo_screenwidth()
    maxh = root.winfo_screenheight()

    # ajuste de las dimensiones y coordenadas de arranque de la ventana...
    root.geometry("%dx%d+%d+%d" % (maxw, maxh, 0, 0))

    # un lienzo de dibujo dentro de la ventana...
    canvas = Canvas(root, width=maxw, height=maxh)
    canvas.grid(column=0, row=0)

    # desarrollar la gráfica...
    button(canvas)

    # lanzar el ciclo principal de control de eventos de la ventana...
    root.mainloop()

if __name__ == '__main__':
    render()
```

¿Cuál de las siguientes versiones de la función **button(canvas)** permitiría dibujar la gráfica sugerida?

Seleccione una:

- ☒ a.

```
def button(canvas):
    x, y, ancho, alto = 100, 100, 100, 25

    # el fondo y el texto del botón...
    canvas.create_rectangle((x, y, x+ancho, y+alto), fill='light gray')
    canvas.create_text(x + ancho//2 - 1, y + alto//2 + 1, text='Ok', fill='black')

    for f in range(2):
        # los bordes blancos...
        canvas.create_line((x+f, y+f, x+ancho-f, y+f), fill='white')
        canvas.create_line((x+f, y+f, x+f, y+alto-f), fill='white')

        # los bordes oscuros...
        canvas.create_line((x+f, y+alto-f, x+ancho-f, y+alto-f), fill='dark gray')
        canvas.create_line((x+ancho-f, y+alto-f, x+ancho-f, y+f), fill='dark gray')
```

✓ ¡Correcto!

☐ b. `def button(canvas):`

```
x, y, ancho, alto = 100, 100, 100, 25

# el fondo y el texto del botón...
canvas.create_rectangle((x, y, x+ancho, y+alto), fill='light gray')
canvas.create_text(x + ancho//2 - 1, y + alto//2 + 1, text='Ok', fill='black')

for f in range(2):
    # los bordes blancos...
    canvas.create_line((x+f, y+f, x+ancho-f, y+f), fill='dark gray')
    canvas.create_line((x+f, y+f, x+f, y+alto-f), fill='dark gray')

    # los bordes oscuros...
    canvas.create_line((x+f, y+alto-f, x+ancho-f, y+alto-f), fill='dark gray')
    canvas.create_line((x+ancho-f, y+alto-f, x+ancho-f, y+f), fill='dark gray')
```

☐ c. `def button(canvas):`

```
x, y, ancho, alto = 100, 100, 100, 25

# el fondo y el texto del botón...
canvas.create_rectangle((x, y, x+ancho, y+alto), fill='light gray')
canvas.create_text(x + ancho//2 - 1, y + alto//2 + 1, text='Exit', fill='black')

for f in range(2):
    # los bordes blancos...
    canvas.create_line((x+f, y+f, x+ancho-f, y+f), fill='white')
    canvas.create_line((x+f, y+f, x+f, y+alto-f), fill='white')

    # los bordes oscuros...
    canvas.create_line((x+f, y+alto-f, x+ancho-f, y+alto-f), fill='dark gray')
    canvas.create_line((x+ancho-f, y+alto-f, x+ancho-f, y+f), fill='dark gray')
```

☐ d. `def button(canvas):`

```
x, y, ancho, alto = 100, 100, 100, 25

# el fondo y el texto del botón...
canvas.create_rectangle((x, y, x+ancho, y+alto), fill='red')
canvas.create_text(x + ancho//2 - 1, y + alto//2 + 1, text='Ok', fill='black')

for f in range(2):
    # los bordes blancos...
    canvas.create_line((x+f, y+f, x+ancho-f, y+f), fill='white')
    canvas.create_line((x+f, y+f, x+f, y+alto-f), fill='white')

    # los bordes oscuros...
    canvas.create_line((x+f, y+alto-f, x+ancho-f, y+alto-f), fill='dark gray')
    canvas.create_line((x+ancho-f, y+alto-f, x+ancho-f, y+f), fill='dark gray')
```

¡Correcto!

La respuesta correcta es:

```
def button(canvas):  
    x, y, ancho, alto = 100, 100, 100, 25  
  
    # el fondo y el texto del botón...  
    canvas.create_rectangle((x, y, x+ancho, y+alto), fill='light gray')  
    canvas.create_text(x + ancho//2 - 1, y + alto//2 + 1, text='Ok', fill='black')  
  
    for f in range(2):  
        # los bordes blancos...  
        canvas.create_line((x+f, y+f, x+ancho-f, y+f), fill='white')  
        canvas.create_line((x+f, y+f, x+f, y+alto-f), fill='white')  
  
        # los bordes oscuros...  
        canvas.create_line((x+f, y+alto-f, x+ancho-f, y+alto-f), fill='dark gray')  
        canvas.create_line((x+ancho-f, y+alto-f, x+ancho-f, y+f), fill='dark gray')
```

Pregunta 14

Correcta

Se puntúa 1 sobre 1

Suponga que se desea generar Python el dibujo *de un campo de edición de textos* para una aplicación que simule una interfaz visual de usuario, como el que se muestra aquí (con las mismas proporciones y colores, aunque sin importar las coordenadas de visualización).



El siguiente programa está pensado para generar esa gráfica, pero dejando la función **edit(canvas)** sin hacer:

```
__author__ = 'Cátedra de AED'

from tkinter import *

def edit(canvas):
    pass

def render():
    # configuracion inicial de la ventana principal...
    root = Tk()
    root.title('Cuestionario')

    # calculo de resolucioen en pixels de la pantalla...
    maxw = root.winfo_screenwidth()
    maxh = root.winfo_screenheight()

    # ajuste de las dimensiones y coordenadas de arranque de la ventana...
    root.geometry("%dx%d+%d+%d" % (maxw, maxh, 0, 0))

    # un lienzo de dibujo dentro de la ventana...
    canvas = Canvas(root, width=maxw, height=maxh)
    canvas.grid(column=0, row=0)

    # desarrollar la gráfica...
    edit(canvas)

    # lanzar el ciclo principal de control de eventos de la ventana...
    root.mainloop()

if __name__ == '__main__':
    render()
```

¿Cuál de las siguientes versiones de la función **edit(canvas)** permitiría dibujar la gráfica sugerida?

Seleccione una:

☒ a.

```
def edit(canvas):
    x, y, ancho, alto = 100, 100, 100, 25

    # el fondo y el texto del campo de edición...
    canvas.create_rectangle((x, y, x+ancho, y+alto), fill='white')
    canvas.create_text(x-25, y+alto//2, text='Valor: ', fill='black')

    for f in range(2):
        # los bordes oscuros...
        canvas.create_line((x+f, y+f, x+ancho-f, y+f), fill='dark gray')
        canvas.create_line((x+f, y+f, x+f, y+alto-f), fill='dark gray')

        # los bordes claros...
        canvas.create_line((x+f, y+alto-f, x+ancho-f, y+alto-f), fill='white')
        canvas.create_line((x+ancho-f, y+alto-f, x+ancho-f, y+f), fill='white')
```

✓ ¡Correcto!

☐ b.

```
def edit(canvas):
    x, y, ancho, alto = 100, 100, 100, 25

    # el fondo y el texto del campo de edición...
    canvas.create_rectangle((x, y, x+ancho, y+alto), fill='white')
    canvas.create_text(x-25, y+alto//2, text='Valor: ', fill='red')

    for f in range(2):
        # los bordes oscuros...
        canvas.create_line((x+f, y+f, x+ancho-f, y+f), fill='dark gray')
        canvas.create_line((x+f, y+f, x+f, y+alto-f), fill='dark gray')

        # los bordes claros...
        canvas.create_line((x+f, y+alto-f, x+ancho-f, y+alto-f), fill='white')
        canvas.create_line((x+ancho-f, y+alto-f, x+ancho-f, y+f), fill='white')
```

☐ c.

```
def edit(canvas):
    x, y, ancho, alto = 100, 100, 100, 25

    # el fondo y el texto del campo de edición...
    canvas.create_rectangle((x, y, x+ancho, y+alto), fill='white')
    canvas.create_text(x-25, y+alto//2, text='Valor: ', fill='black')

    for f in range(2):
        # los bordes oscuros...
        canvas.create_line((x+f, y+f, x+ancho-f, y+f), fill='white')
        canvas.create_line((x+f, y+f, x+f, y+alto-f), fill='white')

        # los bordes claros...
        canvas.create_line((x+f, y+alto-f, x+ancho-f, y+alto-f), fill='dark gray')
        canvas.create_line((x+ancho-f, y+alto-f, x+ancho-f, y+f), fill='dark gray')
```

☐ d.

```
def edit(canvas):
    x, y, ancho, alto = 100, 100, 100, 25

    # el fondo y el texto del campo de edición...
    canvas.create_rectangle((x, y, x+ancho, y+alto), fill='white')
    canvas.create_text(x-25, y+alto//2, text='Valor: ', fill='black')

    for f in range(6):
        # los bordes oscuros...
        canvas.create_line((x+f, y+f, x+ancho-f, y+f), fill='dark gray')
        canvas.create_line((x+f, y+f, x+f, y+alto-f), fill='dark gray')

        # los bordes claros...
        canvas.create_line((x+f, y+alto-f, x+ancho-f, y+alto-f), fill='white')
        canvas.create_line((x+ancho-f, y+alto-f, x+ancho-f, y+f), fill='white')
```

¡Correcto!

La respuesta correcta es:

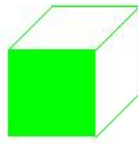

```
def edit(canvas):  
    x, y, ancho, alto = 100, 100, 100, 25  
  
    # el fondo y el texto del campo de edición...  
    canvas.create_rectangle((x, y, x+ancho, y+alto), fill='white')  
    canvas.create_text(x-25, y+alto//2, text='Valor: ', fill='black')  
  
    for f in range(2):  
        # los bordes oscuros...  
        canvas.create_line((x+f, y+f, x+ancho-f, y+f), fill='dark gray')  
        canvas.create_line((x+f, y+f, x+f, y+alto-f), fill='dark gray')  
  
        # los bordes claros...  
        canvas.create_line((x+f, y+alto-f, x+ancho-f, y+alto-f), fill='white')  
        canvas.create_line((x+ancho-f, y+alto-f, x+ancho-f, y+f), fill='white')
```

Pregunta **15**

Incorrecta

Se puntúa 0 sobre 1

Suponga que se desea generar Python *el dibujo de un cubo* como el que se muestra aquí (con las mismas proporciones y colores, aunque sin importar las coordenadas de visualización).



El siguiente programa está pensado para generar esa gráfica, pero dejando la función **cube(canvas)** sin hacer:

```
__author__ = 'Cátedra de AED'

from tkinter import *

def cube(canvas):
    pass

def render():
    # configuracion inicial de la ventana principal...
    root = Tk()
    root.title('Cuestionario')

    # calculo de resolucion en pixels de la pantalla...
    maxw = root.winfo_screenwidth()
    maxh = root.winfo_screenheight()

    # ajuste de las dimensiones y coordenadas de arranque de la ventana...
    root.geometry("%dx%d+%d+%d" % (maxw, maxh, 0, 0))

    # un lienzo de dibujo dentro de la ventana...
    canvas = Canvas(root, bg='white', width=maxw, height=maxh)
    canvas.grid(column=0, row=0)

    # desarrollar la gráfica...
    cube(canvas)

    # lanzar el ciclo principal de control de eventos de la ventana...
    root.mainloop()

if __name__ == '__main__':
    render()
```

¿Cuál de las siguientes versiones de la función **cube(canvas)** permitiría dibujar la gráfica sugerida?

Seleccione una:

- ☒ a. `def cube(canvas):`
`x, y, ancho, alto = 100, 100, 60, 60`
`canvas.create_rectangle((x, y, x+ancho, y+alto), outline='lime', fill='lime')`
`canvas.create_line((100, 100, 130, 70), fill='lime')`
`canvas.create_line((160, 100, 190, 70), fill='lime')`
`canvas.create_line((160, 160, 190, 130), fill='lime')`

✗ Incorrecto...

- ☐ b. _____

```
def cube(canvas):  
    x, y, ancho, alto = 100, 100, 60, 60  
    canvas.create_rectangle((x, y, x+ancho, y+alto), outline='navy', fill='navy')  
    canvas.create_line((100, 100, 130, 70), fill='lime')  
    canvas.create_line((160, 100, 190, 70), fill='lime')  
    canvas.create_line((130, 70, 190, 70), fill='lime')  
    canvas.create_line((190, 70, 190, 130), fill='lime')  
    canvas.create_line((160, 160, 190, 130), fill='lime')
```

☐ c.

```
def cube(canvas):  
    x, y, ancho, alto = 100, 100, 60, 60  
    canvas.create_rectangle((x, y, x+ancho, y+alto), outline='lime', fill='lime')  
    canvas.create_line((100, 100, 130, 70), fill='lime')  
    canvas.create_line((160, 100, 190, 70), fill='lime')  
    canvas.create_line((130, 70, 190, 70), fill='lime')  
    canvas.create_line((190, 70, 190, 130), fill='lime')  
    canvas.create_line((160, 160, 190, 130), fill='lime')
```

☐ d.

```
def cube(canvas):  
    x, y, ancho, alto = 100, 100, 60, 60  
    canvas.create_rectangle((x, y, x+ancho, y+alto), outline='lime', fill='lime', stipple='gray12')  
    canvas.create_line((100, 100, 130, 70), fill='lime')  
    canvas.create_line((160, 100, 190, 70), fill='lime')  
    canvas.create_line((130, 70, 190, 70), fill='lime')  
    canvas.create_line((190, 70, 190, 130), fill='lime')  
    canvas.create_line((160, 160, 190, 130), fill='lime')
```

Incorrecto... revise la Ficha 27, página 553 y siguientes...

La respuesta correcta es:

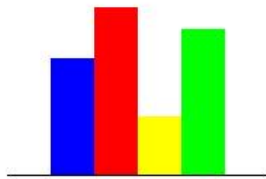
```
def cube(canvas):  
    x, y, ancho, alto = 100, 100, 60, 60  
    canvas.create_rectangle((x, y, x+ancho, y+alto), outline='lime', fill='lime')  
    canvas.create_line((100, 100, 130, 70), fill='lime')  
    canvas.create_line((160, 100, 190, 70), fill='lime')  
    canvas.create_line((130, 70, 190, 70), fill='lime')  
    canvas.create_line((190, 70, 190, 130), fill='lime')  
    canvas.create_line((160, 160, 190, 130), fill='lime')
```

Pregunta 16

Correcta

Se puntúa 1 sobre 1

Suponga que se desea generar en Python un esquema estadístico tipo *diagrama de barras*, como el que se muestra aquí (con las mismas proporciones y colores, aunque sin importar las coordenadas de visualización).



El siguiente programa está pensado para generar esa gráfica, pero dejando la *bars(canvas)* sin hacer:

```
__author__ = 'Cátedra de AED'

from tkinter import *

def bars(canvas):
    pass

def render():
    # configuracion inicial de la ventana principal...
    root = Tk()
    root.title('Cuestionario')

    # calculo de resolucion en pixels de la pantalla...
    maxw = root.winfo_screenwidth()
    maxh = root.winfo_screenheight()

    # ajuste de las dimensiones y coordenadas de arranque de la ventana...
    root.geometry("%dx%d+%d+%d" % (maxw, maxh, 0, 0))

    # un lienzo de dibujo dentro de la ventana...
    canvas = Canvas(root, bg='white', width=maxw, height=maxh)
    canvas.grid(column=0, row=0)

    # desarrollar la gráfica...
    bars(canvas)

    # lanzar el ciclo principal de control de eventos de la ventana...
    root.mainloop()

if __name__ == '__main__':
    render()
```

¿Cuál de las siguientes versiones de la función *bars(canvas)* permitiría dibujar la gráfica sugerida?

Seleccione una:

- ☐ a. `def bars(canvas):`
`canvas.create_rectangle((100, 120, 130, 200), outline='red', fill='red')`
`canvas.create_rectangle((130, 85, 160, 200), outline='yellow', fill='yellow')`
`canvas.create_rectangle((161, 160, 190, 200), outline='lime', fill='lime')`
`canvas.create_rectangle((190, 100, 220, 200), outline='blue', fill='blue')`
`canvas.create_line((70, 200, 250, 200), fill='black')`

- ☐ b.

```
def bars(canvas):  
    canvas.create_rectangle((100, 120, 130, 200), outline='blue')  
    canvas.create_rectangle((130, 85, 160, 200), outline='red')  
    canvas.create_rectangle((161, 160, 190, 200), outline='yellow')  
    canvas.create_rectangle((190, 100, 220, 200), outline='lime')  
    canvas.create_line((70, 200, 250, 200), fill='black')
```

☐

c.

```
def bars(canvas):  
    canvas.create_rectangle((100, 120, 130, 200), outline='blue', fill='blue')  
    canvas.create_rectangle((130, 120, 160, 200), outline='red', fill='red')  
    canvas.create_rectangle((161, 120, 190, 200), outline='yellow', fill='yellow')  
    canvas.create_rectangle((190, 120, 220, 200), outline='lime', fill='lime')  
    canvas.create_line((70, 200, 250, 200), fill='black')
```

☒

d.

```
def bars(canvas):  
    canvas.create_rectangle((100, 120, 130, 200), outline='blue', fill='blue')  
    canvas.create_rectangle((130, 85, 160, 200), outline='red', fill='red')  
    canvas.create_rectangle((161, 160, 190, 200), outline='yellow', fill='yellow')  
    canvas.create_rectangle((190, 100, 220, 200), outline='lime', fill='lime')  
    canvas.create_line((70, 200, 250, 200), fill='black')
```



¡Correcto!

¡Correcto!

La respuesta correcta es:

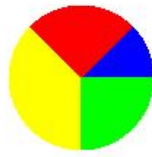
```
def bars(canvas):  
    canvas.create_rectangle((100, 120, 130, 200), outline='blue', fill='blue')  
    canvas.create_rectangle((130, 85, 160, 200), outline='red', fill='red')  
    canvas.create_rectangle((161, 160, 190, 200), outline='yellow', fill='yellow')  
    canvas.create_rectangle((190, 100, 220, 200), outline='lime', fill='lime')  
    canvas.create_line((70, 200, 250, 200), fill='black')
```

Pregunta **17**

Correcta

Se puntúa 1 sobre 1

Suponga que se desea generar en Python un esquema estadístico tipo *diagrama de segmentos circulares*, como el que se muestra aquí (con las mismas proporciones y colores, aunque sin importar las coordenadas de visualización).



El siguiente programa está pensado para generar esa gráfica, pero dejando la *pie(canvas)* sin hacer:

```
__author__ = 'Cátedra de AED'

from tkinter import *

def pie(canvas):
    pass

def render():
    # configuracion inicial de la ventana principal...
    root = Tk()
    root.title('Cuestionario')

    # calculo de resolucioen en pixels de la pantalla...
    maxw = root.winfo_screenwidth()
    maxh = root.winfo_screenheight()

    # ajuste de las dimensiones y coordenadas de arranque de la ventana...
    root.geometry("%dx%d+%d+%d" % (maxw, maxh, 0, 0))

    # un lienzo de dibujo dentro de la ventana...
    canvas = Canvas(root, bg='white', width=maxw, height=maxh)
    canvas.grid(column=0, row=0)

    # desarrollar la gráfica...
    pie(canvas)

    # lanzar el ciclo principal de control de eventos de la ventana...
    root.mainloop()

if __name__ == '__main__':
    render()
```

¿Cuál de las siguientes versiones de la función *pie(canvas)* permitiría dibujar la gráfica sugerida?

Seleccione una:

☐ a.

```
def pie(canvas):
    x, y, ancho, alto = 100, 100, 100, 100
    canvas.create_arc((x, y, x+ancho, y+alto), start=0, extent=45, outline='blue', style=PIESLICE)
    canvas.create_arc((x, y, x+ancho, y+alto), start=45, extent=90, outline='red', style=PIESLICE)
    canvas.create_arc((x, y, x+ancho, y+alto), start=135, extent=135, outline='yellow', style=PIESLICE)
    canvas.create_arc((x, y, x+ancho, y+alto), start=270, extent=90, outline='lime', style=PIESLICE)
```

☐ b.

```
def pie(canvas):
    x, y, ancho, alto = 100, 100, 300, 100
    canvas.create_arc((x, y, x+ancho, y+alto), start=0, extent=45, outline='blue', fill='blue', style=PIESLICE)
    canvas.create_arc((x, y, x+ancho, y+alto), start=45, extent=90, outline='red', fill='red', style=PIESLICE)
    canvas.create_arc((x, y, x+ancho, y+alto), start=135, extent=135, outline='yellow', fill='yellow', style=PIESLICE)
    canvas.create_arc((x, y, x+ancho, y+alto), start=270, extent=90, outline='lime', fill='lime', style=PIESLICE)
```

☐ c. `def pie(canvas):`
 `x, y, ancho, alto = 100, 100, 100, 100`
 `canvas.create_arc((x, y, x+ancho, y+alto), start=0, extent=45, outline='blue', fill='blue', style=ARC)`
 `canvas.create_arc((x, y, x+ancho, y+alto), start=45, extent=90, outline='red', fill='red', style=ARC)`
 `canvas.create_arc((x, y, x+ancho, y+alto), start=135, extent=135, outline='yellow', fill='yellow', style=ARC)`
 `canvas.create_arc((x, y, x+ancho, y+alto), start=270, extent=90, outline='lime', fill='lime', style=ARC)`

☒ d. `def pie(canvas):`
 `x, y, ancho, alto = 100, 100, 100, 100`
 `canvas.create_arc((x, y, x+ancho, y+alto), start=0, extent=45, outline='blue', fill='blue',`
 `style=PIESLICE)`
 `canvas.create_arc((x, y, x+ancho, y+alto), start=45, extent=90, outline='red', fill='red',`
 `style=PIESLICE)`
 `canvas.create_arc((x, y, x+ancho, y+alto), start=135, extent=135, outline='yellow', fill='yellow',`
 `style=PIESLICE)`
 `canvas.create_arc((x, y, x+ancho, y+alto), start=270, extent=90, outline='lime', fill='lime',`
 `style=PIESLICE)`

✓ ¡Correcto!

¡Correcto!

La respuesta correcta es:

```
def pie(canvas):
    x, y, ancho, alto = 100, 100, 100, 100
    canvas.create_arc((x, y, x+ancho, y+alto), start=0, extent=45, outline='blue', fill='blue', style=PIESLICE)
    canvas.create_arc((x, y, x+ancho, y+alto), start=45, extent=90, outline='red', fill='red', style=PIESLICE)
    canvas.create_arc((x, y, x+ancho, y+alto), start=135, extent=135, outline='yellow', fill='yellow', style=PIESLICE)
    canvas.create_arc((x, y, x+ancho, y+alto), start=270, extent=90, outline='lime', fill='lime', style=PIESLICE)
```

Pregunta 18

Correcta

Se puntúa 1 sobre 1

Suponga que se desea generar en Python el dibujo del *esquema básico de una cara*, como el que se muestra aquí (con las mismas proporciones y colores, aunque sin importar las coordenadas de visualización).



El siguiente programa está pensado para generar esa gráfica, pero dejando la *face(canvas)* sin hacer:

```
__author__ = 'Cátedra de AED'

from tkinter import *

def face(canvas):
    pass

def render():
    # configuracion inicial de la ventana principal...
    root = Tk()
    root.title('Cuestionario')

    # calculo de resolucion en pixels de la pantalla...
    maxw = root.winfo_screenwidth()
    maxh = root.winfo_screenheight()

    # ajuste de las dimensiones y coordenadas de arranque de la ventana...
    root.geometry("%dx%d+%d+%d" % (maxw, maxh, 0, 0))

    # un lienzo de dibujo dentro de la ventana...
    canvas = Canvas(root, bg='white', width=maxw, height=maxh)
    canvas.grid(column=0, row=0)

    # desarrollar la gráfica...
    face(canvas)

    # lanzar el ciclo principal de control de eventos de la ventana...
    root.mainloop()

if __name__ == '__main__':
    render()
```

¿Cuál de las siguientes versiones de la función *face(canvas)* permitiría dibujar la gráfica sugerida?

Seleccione una:

☐ a.

```
def face(canvas):
    canvas.create_oval((100, 100, 160, 160), outline='red', fill='pink')
    canvas.create_oval((110, 115, 125, 125), outline='blue', fill='blue')
    canvas.create_oval((135, 115, 150, 125), outline='blue', fill='blue')
    canvas.create_line((127, 134, 133, 134), fill='blue')
    canvas.create_arc((120, 138, 140, 148), start=180, extent=180, outline='blue', style=ARC)
```

☒ b.

```
def face(canvas):
    canvas.create_oval((100, 100, 160, 160), outline='red')
    canvas.create_oval((110, 115, 125, 125), outline='blue', fill='blue')
    canvas.create_oval((135, 115, 150, 125), outline='blue', fill='blue')
    canvas.create_line((127, 134, 133, 134), fill='blue')
    canvas.create_arc((120, 138, 140, 148), start=180, extent=180, outline='blue', style=ARC)
```

✓ ¡Correcto!

☐ c.

```
def face(canvas):  
    canvas.create_oval((100, 100, 160, 160), outline='red')  
    canvas.create_oval((110, 115, 125, 125), outline='blue', fill='blue')  
    canvas.create_line((127, 134, 133, 134), fill='blue')  
    canvas.create_arc((120, 138, 140, 148), start=180, extent=180, outline='blue', style=ARC)
```

☐ d.

```
def face(canvas):  
    canvas.create_oval((100, 100, 160, 160), outline='red')  
    canvas.create_oval((110, 115, 125, 125), outline='blue', fill='blue')  
    canvas.create_oval((135, 115, 150, 125), outline='blue', fill='blue')  
    canvas.create_line((127, 134, 133, 134), fill='blue')  
    canvas.create_arc((120, 138, 140, 148), start=0, extent=180, outline='blue', style=ARC)
```

¡Correcto!

La respuesta correcta es:

```
def face(canvas):  
    canvas.create_oval((100, 100, 160, 160), outline='red')  
    canvas.create_oval((110, 115, 125, 125), outline='blue', fill='blue')  
    canvas.create_oval((135, 115, 150, 125), outline='blue', fill='blue')  
    canvas.create_line((127, 134, 133, 134), fill='blue')  
    canvas.create_arc((120, 138, 140, 148), start=180, extent=180, outline='blue', style=ARC)
```

Pregunta 19

Correcta

Se puntúa 1 sobre 1

Suponga que se desea generar en Python el *dibujo del popular Pacman*, como el que se muestra aquí (con las mismas proporciones y colores, aunque sin importar las coordenadas de visualización).



El siguiente programa está pensado para generar esa gráfica, pero dejando la *pacman(canvas)* sin hacer:

```
__author__ = 'Cátedra de AED'

from tkinter import *

def pacman(canvas):
    pass

def render():
    # configuracion inicial de la ventana principal...
    root = Tk()
    root.title('Cuestionario')

    # calculo de resolucioen en pixels de la pantalla...
    maxw = root.winfo_screenwidth()
    maxh = root.winfo_screenheight()

    # ajuste de las dimensiones y coordenadas de arranque de la ventana...
    root.geometry("%dx%d+%d+%d" % (maxw, maxh, 0, 0))

    # un lienzo de dibujo dentro de la ventana...
    canvas = Canvas(root, bg='white', width=maxw, height=maxh)
    canvas.grid(column=0, row=0)

    # desarrollar la gráfica...
    pacman(canvas)

    # lanzar el ciclo principal de control de eventos de la ventana...
    root.mainloop()

if __name__ == '__main__':
    render()
```

¿Cuál de las siguientes versiones de la función *pacman(canvas)* permitiría dibujar la gráfica sugerida?

Seleccione una:

- ☐ a.

```
def pacman(canvas):
    x, y, ancho, alto = 100, 100, 50, 50
    canvas.create_arc((x, y, x+ancho, y+alto), start=45, extent=359, outline='lime', fill='lime', style=PIESLICE)
    canvas.create_oval((x+10, y+10, x+2+ancho//3, y+2+alto//3), outline='white', fill='white')
    canvas.create_oval((x+16, y+16, x+ancho/4, y+alto/4), outline='black', fill='black')
```
- ☐ b.

```
def pacman(canvas):
    x, y, ancho, alto = 100, 100, 50, 50
    canvas.create_arc((x, y, x+ancho, y+alto), start=45, extent=315, outline='red', fill='red', style=PIESLICE)
    canvas.create_oval((x+10, y+10, x+2+ancho//3, y+2+alto//3), outline='white', fill='white')
    canvas.create_oval((x+16, y+16, x+ancho/4, y+alto/4), outline='black', fill='black')
```
- ☐ c.

```
def pacman(canvas):  
    x, y, ancho, alto = 100, 100, 50, 50  
    canvas.create_arc((x, y, x+ancho, y+alto), start=45, extent=315, outline='lime', fill='lime', style=PIESLICE)  
    canvas.create_oval((x+10, y+10, x+2+ancho//3, y+2+alto//3), outline='red', fill='red')  
    canvas.create_oval((x+16, y+16, x+ancho/4, y+alto/4), outline='black', fill='black')
```

☐ d.

```
def pacman(canvas):  
    x, y, ancho, alto = 100, 100, 50, 50  
    canvas.create_arc((x, y, x+ancho, y+alto), start=45, extent=315, outline='lime', fill='lime',  
style=PIESLICE)  
    canvas.create_oval((x+10, y+10, x+2+ancho//3, y+2+alto//3), outline='white', fill='white')  
    canvas.create_oval((x+16, y+16, x+ancho/4, y+alto/4), outline='black', fill='black')
```

✓ ¡Correcto!

¡Correcto!

La respuesta correcta es:

```
def pacman(canvas):  
    x, y, ancho, alto = 100, 100, 50, 50  
    canvas.create_arc((x, y, x+ancho, y+alto), start=45, extent=315, outline='lime', fill='lime', style=PIESLICE)  
    canvas.create_oval((x+10, y+10, x+2+ancho//3, y+2+alto//3), outline='white', fill='white')  
    canvas.create_oval((x+16, y+16, x+ancho/4, y+alto/4), outline='black', fill='black')
```

[◀ Materiales Adicionales para la Ficha 14](#)

Ir a...

[Simulacro 1 para el Parcial 2 - Enunciado ▶](#)

[Página Principal](#) / [Mis cursos](#) / [AED-2024-IntLib-\(Feb2025\)](#) / [Ficha 16](#) / [Cuestionario 16 \[Temas: hasta Ficha 16\]](#)

Comenzado el	miércoles, 5 de marzo de 2025, 13:29
Estado	Finalizado
Finalizado en	miércoles, 5 de marzo de 2025, 14:07
Tiempo empleado	37 minutos 54 segundos
Puntos	13/13
Calificación	10 de 10 (100%)

Pregunta **1**

Correcta

Se puntúa 1 sobre 1

¿Cuál es la **idea principal** en la que se basa el algoritmo de ordenamiento de **Selección Directa** que se presentó en clase? (Suponga que se desea ordenar el arreglo de menor a mayor).

Seleccione una:

- ☒ a. Realizar varias pasadas sobre el arreglo, en cada pasada **seleccionar el menor valor** de los que aún no han sido **✓ ¡Correcto!** ordenados, y colocarlo en su casilla definitiva.
- ☐ b. Realizar varias pasadas sobre el arreglo, en cada pasada **comparar a cada elemento con el que le sigue inmediatamente**, intercambiarlos si están desordenados y seguir haciendo pasadas en esta forma hasta que el arreglo quede ordenado.
- ☐ c. Realizar varias pasadas sobre el arreglo. En cada pasada seleccionar un elemento cualquiera x. Pasar a la derecha del arreglo todos los valores mayores a x. Pasar a la izquierda del arreglo todos los valores menores a x. Finalmente, aplicar la misma idea a cada una de las dos "mitades" así obtenidas y seguir así hasta que ya no pueda seguir obteniendo nuevas "mitades".
- ☐ d. Realizar **una sola pasada sobre el arreglo**, en esa pasada comparar a cada elemento con el que le sigue e intercambiarlos si están invertidos.

¡Correcto!

La respuesta correcta es:

Realizar varias pasadas sobre el arreglo, en cada pasada **seleccionar el menor valor** de los que aún no han sido ordenados, y colocarlo en su casilla definitiva.

Pregunta 2

Correcta

Se puntúa 1 sobre 1

La siguiente función implementa el algoritmo de ordenamiento por *Selección Directa*, tal como se analizó en clases para ordenar de menor a mayor un arreglo v con n componentes:

```
def selection_sort(v):  
    # ordenamiento por seleccion directa  
    n = len(v)  
    for i in range(n-1):  
        for j in range(i+1, n):  
            if v[i] > v[j]:  
                v[i], v[j] = v[j], v[i]
```

¿Qué efecto se produciría en la función anterior si la instrucción condicional `if v[i] > v[j]:` fuese reemplaza por `if v[i] < v[j]:`?

Seleccione una:

- ☐ a. Provocará que el programa se interrumpa con un mensaje de error en la primera comparación.
- ☒ b. El arreglo sería ordenado, pero ahora de mayor a menor. ✓ ¡Correcto!
- ☐ c. No causaría ningún efecto particular: el arreglo seguiría siendo ordenado de menor a mayor.
- ☐ d. Provocaría que el arreglo permanezca siempre sin cambio alguno (no será nunca ordenado de forma alguna, ni se modificará nunca su contenido original).

¡Correcto!

La respuesta correcta es:

El arreglo sería ordenado, pero ahora de mayor a menor.

Pregunta 3

Correcta

Se puntúa 1 sobre 1

¿Cuántas comparaciones hace el algoritmo de Búsqueda Secuencial para encontrar un valor x en un arreglo de n componentes en el *peor caso posible*? (El peor caso es el que obliga a un algoritmo a hacer la máxima cantidad de trabajo. Obviamente, en el caso de la Búsqueda Secuencial ese peor caso se presenta si el valor buscado está exactamente en la última casilla, o bien, si el valr buscado no está en el arreglo).

Seleccione una:

- ☐ a. $\log_2(n)$ comparaciones.
- ☐ b. n^2 comparaciones.
- ☐ c. Una sola comparación, siempre.
- ☒ d. n comparaciones. ✓ ¡Correcto!

¡Correcto!

La respuesta correcta es:

n comparaciones.

Pregunta 4

Correcta

Se puntúa 2 sobre 2

Oportunamente se presentó en clases el algoritmo de **búsqueda secuencial**, el cual toma un arreglo, busca un valor x en el mismo casilla por casilla, y retorna el índice de la casilla que lo contiene (si x está en el arreglo) o retorna -1 si x no está en el arreglo. Suponga que propone la siguiente variante para el algoritmo de búsqueda secuencial:

```
def linear_search(v, x):  
    r = -1  
    for i in range(len(v)):  
        if x == v[i]:  
            r = i  
  
    return r
```

¿Hay algún inconveniente o problema con esta variante?

Seleccione una:

- ☒ a. Hay un inconveniente: la variante propuesta encuentra el valor buscado (si existía) pero no corta el ciclo de búsqueda al encontrarlo con lo que siempre se hacen n comparaciones. ✓ ¡Correcto!
- ☐ b. La variante propuesta funciona correctamente.
- ☐ c. La variante propuesta no funciona correctamente: sólo llega a analizar el contenido de la primera casilla.
- ☐ d. La variante propuesta funciona solamente si el valor x está en el arreglo (falla si x no está).

¡Correcto!

La respuesta correcta es:

Hay un inconveniente: la variante propuesta encuentra el valor buscado (si existía) pero no corta el ciclo de búsqueda al encontrarlo con lo que siempre se hacen n comparaciones.

Pregunta 5

Correcta

Se puntúa 2 sobre 2

Oportunamente se presentó en clases el algoritmo de **búsqueda secuencial**, el cual toma un arreglo, busca un valor x en el mismo casilla por casilla, y retorna el índice de la casilla que lo contiene (si x está en el arreglo) o retorna -1 si x no está en el arreglo. Suponga que propone la siguiente variante para el algoritmo de búsqueda secuencial:

```
def linear_search(v, x):  
    n = len(v)  
    for i in range(n):  
        if x == v[i]:  
            return i  
    else:  
        return -1
```

¿Funciona correctamente esta variante? Si no funciona, ¿cuál es el problema?

Seleccione una:

- ☐ a. La variante propuesta funciona solamente si el valor x está en el arreglo (falla si x no está).
- ☐ b. La variante propuesta funciona correctamente.
- ☒ c. La variante propuesta no funciona correctamente: sólo llega a analizar el contenido de la primera casilla. ✓ ¡Correcto!
- ☐ d. La variante propuesta provoca un error de intérprete: un **if** no puede tener una instrucción **return** en cada una de sus ramas.

¡Correcto!

La respuesta correcta es:

La variante propuesta no funciona correctamente: sólo llega a analizar el contenido de la primera casilla.

Pregunta 6

Correcta

Se puntúa 1 sobre 1

¿En cuáles de los siguientes casos es aplicable el algoritmo de **búsqueda binaria** en un arreglo? (Seleccione todas las respuestas que considere correctas)

Seleccione una o más de una:

- ☐ a. El arreglo en el cual se debe realizar la búsqueda está desordenado y no se nos permite ordenarlo.
- ☒ b. El arreglo en el cual se debe realizar la búsqueda está desordenado, se nos permite ordenarlo, luego permanecerá sin cambios y debemos realizar muchas búsquedas. ✓ ¡Correcto!
- ☐ c. El arreglo en el cual se debe realizar la búsqueda está desordenado, se nos permite ordenarlo, luego podrá alterarse el contenido (quedando eventualmente desordenado) y debemos realizar varias búsquedas.
- ☒ d. El arreglo en el cual se debe realizar la búsqueda está ordenado y permanecerá sin cambios. ✓ ¡Correcto!

¡Correcto!

Las respuestas correctas son:

El arreglo en el cual se debe realizar la búsqueda está ordenado y permanecerá sin cambios,

El arreglo en el cual se debe realizar la búsqueda está desordenado, se nos permite ordenarlo, luego permanecerá sin cambios y debemos realizar muchas búsquedas.

Pregunta 7

Correcta

Se puntúa 2 sobre 2

Oportunamente se presentó en clases el algoritmo de **búsqueda binaria**, el cual toma un arreglo ordenado, busca un valor x en el mismo, y retorna el índice de la casilla que lo contiene (si x está en el arreglo) o retorna -1 si x no está en el arreglo. Suponga que propone la siguiente variante para el algoritmo de búsqueda binaria:

```
def binary_search(v, x):  
    # busqueda binaria... asume arreglo ordenado...  
    izq, der = 0, len(v) - 1  
    while izq <= der:  
        c = (izq + der) // 2  
        if x == v[c]:  
            return c  
        else:  
            return -1  
  
        if x < v[c]:  
            der = c - 1  
        else:  
            izq = c + 1  
  
    return -1
```

¿Funciona correctamente esta variante? Si no funciona, ¿cuál es el problema?

Seleccione una:

- ☒ a. La variante propuesta no funciona correctamente: sólo llega a analizar el contenido de la casilla del centro del arreglo. ✓ ¡Correcto!
- ☐ b. La variante propuesta provoca un error de intérprete y no llega a arrancar: el segundo **if** incluido dentro del ciclo nunca puede ejecutarse, ya que las dos ramas del **if** anterior terminan con una instrucción **return**.
- ☐ c. La variante propuesta funciona solamente si el valor x está en el arreglo (falla si x no está).
- ☐ d. La variante propuesta funciona correctamente.

¡Correcto!

La respuesta correcta es:

La variante propuesta no funciona correctamente: sólo llega a analizar el contenido de la casilla del centro del arreglo.

Pregunta 8

Correcta

Se puntúa 1 sobre 1

Oportunamente se presentó en clases el algoritmo de **búsqueda binaria**, el cual toma un arreglo ordenado, busca un valor x en el mismo, y retorna el índice de la casilla que lo contiene (si x está en el arreglo) o retorna -1 si x no está en el arreglo. Mostramos el algoritmo para dar mejor contexto a la pregunta:

```
def binary_search(v, x):  
    # búsqueda binaria... asume arreglo ordenado...  
    izq, der = 0, len(v) - 1  
    while izq <= der:  
        c = (izq + der) // 2  
        if x == v[c]:  
            return c  
        if x < v[c]:  
            der = c - 1  
        else:  
            izq = c + 1  
  
    return -1
```

¿Qué pasaría con la función anterior si el arreglo v contuviese *cadena de caracteres* en cada casillero (en lugar de números), y el parámetro x contuviese también una cadena de caracteres?

Seleccione una:

- ☐ a. La función mostrada no provocaría que el programa se interrumpa, pero funcionaría en forma incorrecta y retornaría siempre -1 (nunca encontraría la cadena x buscada).
- ☐ b. La función mostrada provocaría un error en tiempo de ejecución y se interrumpiría el programa al intentar determinar si x menor o mayor que $v[i]$.
- ☐ c. La función mostrada funciona solamente si el valor x está en el arreglo (falla si x no está, interrumpiendo el programa con un mensaje de error).
- ☒ d. La función mostrada funcionaría correctamente de todos modos. ✓ ¡Correcto!

¡Correcto!

La respuesta correcta es:

La función mostrada funcionaría correctamente de todos modos.

Pregunta **9**

Correcta

Se puntúa 1 sobre 1

¿Qué pasaría con el algoritmo de *Búsqueda Binaria* si el valor buscado x estuviese repetido más de una vez en el arreglo?

Seleccione una:

- ☐ a. El algoritmo no provocaría que el programa se interrumpa, pero funcionaría en forma incorrecta y retornaría siempre -1 (nunca encontraría *el valor x buscado*).
- ☐ b. El algoritmo funciona solamente si el valor x está en el arreglo (falla si x no está, interrumpiendo el programa con un mensaje de error).
- ☒ c. El algoritmo funcionaría correctamente de todos modos: retornaría el índice de la primera casilla encontrada que contenga a x . ✓ ¡Correcto!
- ☐ d. El algoritmo produciría un error en tiempo de ejecución y se interrumpiría el programa.

¡Correcto!

La respuesta correcta es:

El algoritmo funcionaría correctamente de todos modos: retornaría el índice de la primera casilla encontrada que contenga a x .

Pregunta **10**

Correcta

Se puntúa 1 sobre 1

¿En cuáles de los siguientes casos es aplicable el algoritmo de *Fusión de Arreglos* que se describió en las fichas de clases para producir un tercer arreglo ordenado? (Seleccione todas las respuestas que considere correctas)

Seleccione una o más de una:

- ☐ a. Uno de los arreglos originales debe estar ordenado y el otro arreglo puede estar desordenado.
- ☐ b. El proceso es aplicable sin importar si los dos arreglos originales están ordenados o no, ya que el algoritmo de fusión analizado primero ordena esos dos arreglos, y luego procede a la fusión.
- ☐ c. Los arreglos originales deben estar desordenados.
- ☒ d. Los arreglos originales deben estar ordenados de menor a mayor si se quiere producir un tercer arreglo ordenado de menor a mayor. ✓ ¡Correcto!

¡Correcto!

La respuesta correcta es:

Los arreglos originales deben estar ordenados de menor a mayor si se quiere producir un tercer arreglo ordenado de menor a mayor.

◀ Materiales Adicionales para la Ficha 16

Ir a...

Guía 16 de Ejercicios Prácticos ►

Comenzado el	miércoles, 5 de marzo de 2025, 14:09
Estado	Finalizado
Finalizado en	miércoles, 5 de marzo de 2025, 20:14
Tiempo empleado	6 horas 5 minutos
Calificación	10 de 10 (100%)

Pregunta 1

Correcta

Se puntúa 1 sobre 1

¿Cuál es la principal característica de todos los métodos de ordenamiento conocidos como métodos simples o directos?

Seleccione una:

- ☐ a. Tienen muy buen rendimiento en tiempo de ejecución, cualquiera sea el tamaño n del arreglo.
- ☒ b. Tienen muy mal rendimiento en tiempo de ejecución si el tamaño n del arreglo es grande o muy grande, y un rendimiento aceptable si n es pequeño. ✓ ¡Correcto!
- ☐ c. Tienen muy mal rendimiento en tiempo de ejecución, cualquiera sea el tamaño n del arreglo.
- ☐ d. Tienen muy mal rendimiento en tiempo de ejecución si el tamaño n del arreglo es pequeño, y un rendimiento aceptable si n es grande o muy grande.

¡Correcto!

La respuesta correcta es:

Tienen muy mal rendimiento en tiempo de ejecución si el tamaño n del arreglo es grande o muy grande, y un rendimiento aceptable si n es pequeño.

Pregunta 2

Correcta

Se puntúa 1 sobre 1

¿Cuál de las siguientes describe mejor la idea de funcionamiento en la que está basado el algoritmo conocido como ordenamiento por *Inserción Simple* para ordenar un arreglo de n componentes?

Seleccione una:

- ☐ a. Reacomodar los n elementos del arreglo en forma aleatoria, controlar si quedó ordenado, y en caso de negativo, volver a reacomodarlos en forma aleatoria, continuando así hasta que en algún momento se obtenga un arreglo ordenado...
- ☒ b. Suponer que el arreglo tiene un subconjunto inicialmente ordenado que contiene sólo al primer elemento, luego realizar n pasadas, de forma que en cada una agregue el siguiente elemento al grupo que está ordenado. ✓ ¡Correcto!
- ☐ c. Realizar n pasadas, de forma que en cada una se compare a cada elemento con el siguiente, logrando que en cada pasada los mayores vayan acomodándose al final del arreglo.
- ☐ d. Realizar n pasadas, de forma que en cada una se determine el menor de los elementos analizados, y llevar ese menor a la casilla pivot.

¡Correcto!

La respuesta correcta es:

Suponer que el arreglo tiene un subconjunto inicialmente ordenado que contiene sólo al primer elemento, luego realizar n pasadas, de forma que en cada una agregue el siguiente elemento al grupo que está ordenado.

Pregunta 3

Correcta

Se puntúa 1 sobre 1

¿Cuál de las siguientes describe mejor la idea de funcionamiento en la que está basado el algoritmo conocido como ordenamiento por *Intercambio Directo* u *Ordenamiento de Burbuja* para ordenar *de menor a mayor* un arreglo de n componentes?

Seleccione una:

- ☐ a. Reacomodar los n elementos del arreglo en forma aleatoria, controlar si quedó ordenado, y en caso negativo, volver a reacomodarlos en forma aleatoria, continuando así hasta que en algún momento se obtenga un arreglo ordenado...
- ☐ b. Realizar n pasadas, de forma que en cada una se determine el menor de los elementos analizados, y llevar ese menor a la casilla pivot.
- ☒ c. Realizar n pasadas, de forma que en cada una se compare a cada elemento con el siguiente, logrando que en cada pasada los mayores vayan acomodándose al final del arreglo. ✓ ¡Correcto!
- ☐ d. Suponer que el arreglo tiene un subconjunto inicialmente ordenado que contiene sólo al primer elemento, luego realizar n pasadas, de forma que en cada una agregue el siguiente elemento al grupo que está ordenado.

¡Correcto!

La respuesta correcta es:

Realizar n pasadas, de forma que en cada una se compare a cada elemento con el siguiente, logrando que en cada pasada los mayores vayan acomodándose al final del arreglo.

Pregunta 4

Correcta

Se puntúa 1 sobre 1

¿Cuál de las siguientes describe mejor la idea de funcionamiento en la que está basado el algoritmo conocido como ordenamiento por *Selección Simple* o *Selección Directa* para ordenar un arreglo de n componentes?

Seleccione una:

- ☐ a. Suponer que el arreglo tiene un subconjunto inicialmente ordenado que contiene sólo al primer elemento, luego realizar n pasadas, de forma que en cada una agregue el siguiente elemento al grupo que está ordenado.
- ☐ b. Realizar n pasadas, de forma que en cada una se compare a cada elemento con el siguiente, logrando que en cada pasada los mayores vaya acomodándose al final del arreglo.
- ☐ c. Reacomodar los n elementos del arreglo en forma aleatoria, controlar si quedó ordenado, y en caso de negativo, volver a reacomodarlos en forma aleatoria, continuando así hasta que en algún momento se obtenga un arreglo ordenado...
- ☒ d. Realizar n pasadas, de forma que en cada una se determine el menor de los elementos analizados, y llevar ese menor a la casilla pivot. ✓ ¡Correcto!

¡Correcto!

La respuesta correcta es:

Realizar n pasadas, de forma que en cada una se determine el menor de los elementos analizados, y llevar ese menor a la casilla pivot.

Pregunta 5

Correcta

Se puntúa 1 sobre 1

¿Por qué motivo el algoritmo Bubblesort para ordenamiento de un arreglo usa una *bandera de corte* en la versión presentada en las fichas de clase?

Seleccione una:

- ☐ a. La bandera de corte se usa para determinar si el ordenamiento debe hacerse de menor a mayor (bandera = True) o de mayor a menor (bandera = False)
- ☐ b. No es cierto que la versión vista en clases use una bandera de corte.
- ☐ c. La bandera de corte se usa para garantizar que el arreglo quede ordenado.
- ☒ d. La bandera de corte se usa para terminar el proceso apenas se detecte que en la pasada actual no hubo intercambios, para ahorrar tiempo. ✓ ¡Correcto!

¡Correcto!

La respuesta correcta es:

La bandera de corte se usa para terminar el proceso apenas se detecte que en la pasada actual no hubo intercambios, para ahorrar tiempo.

Pregunta **6**

Correcta

Se puntúa 1 sobre 1

¿Cuál de los siguientes es el creador del famoso algoritmo de ordenamiento conocido como **Quicksort**?

Seleccione una:

- ☒ a. **Charles Antony Richard Hoare** ✓ ¡Correcto!
- ☐ b. Edsger Wybe Dijkstra
- ☐ c. J. W. J. Williams
- ☐ d. Donald Shell

¡Correcto!

La respuesta correcta es:

Charles Antony Richard Hoare

Pregunta **7**

Correcta

Se puntúa 1 sobre 1

¿Cuáles de las siguientes son características **correctas** del algoritmo *Shellsort*? (Más de una puede ser cierta... marque TODAS las que considere válidas)

Seleccione una o más de una:

- ☐ a. El algoritmo Shellsort consiste en una mejora del algoritmo de Selección Directa, basada en buscar iterativamente el menor (o el mayor) entre los elementos que quedan en el vector, para llevarlo a su posición correcta, pero de forma que la búsqueda del menor en cada vuelta se haga en tiempo logarítmico.
- ☒ b. **El algoritmo Shellsort es complejo de analizar para determinar su rendimientos en forma matemática, ya que ese rendimiento depende fuertemente de la serie de incrementos decreciente que se haya seleccionado.** ✓ ¡Correcto!
- ☒ c. **El algoritmo Shellsort consiste en una mejora del algoritmo de Inserción Directa (o Inserción Simple), consistente en armar subconjuntos ordenados con elementos a distancia $h > 1$ en las primeras fases, y terminar con $h = 1$ en la última.** ✓ ¡Correcto!
- ☐ d. Una muy buena serie de incrementos decrecientes a usar, es la serie $h = \{...16, 8, 4, 2, 1\}$

¡Correcto!

Las respuestas correctas son:

El algoritmo Shellsort consiste en una mejora del algoritmo de Inserción Directa (o Inserción Simple), consistente en armar subconjuntos ordenados con elementos a distancia $h > 1$ en las primeras fases, y terminar con $h = 1$ en la última.,

El algoritmo Shellsort es complejo de analizar para determinar su rendimientos en forma matemática, ya que ese rendimiento depende fuertemente de la serie de incrementos decreciente que se haya seleccionado.

Pregunta 8

Correcta

Se puntúa 1 sobre 1

¿Cuál es el problema si en el algoritmo *Shellsort* se elige una serie de incrementos decrecientes de la forma { ..., 16, 8, 4, 2, 1 } ?

Seleccione una:

- ☐ a. No sólo no hay ningún problema, sino que esa serie es la mejor posible para el algoritmo Shellsort.
- ☐ b. Ningún problema: esa serie es tan buena como cualquier otra.
- ☐ c. El arreglo no quedará ordenado al final.
- ☒ d. Los subconjuntos analizados contendrán casi los mismos elementos cuando la distancia usada sea cada vez menor, sin garantías de lograr una buena organización del arreglo antes de la última pasada. ✓ ¡Correcto!

¡Correcto!

La respuesta correcta es:

Los subconjuntos analizados contendrán casi los mismos elementos cuando la distancia usada sea cada vez menor, sin garantías de lograr una buena organización del arreglo antes de la última pasada.

Pregunta 9

Correcta

Se puntúa 1 sobre 1

Sabemos que en el *algoritmo de Shell* se termina haciendo una última pasada sobre el arreglo con incremento de compración $h = 1$ ¿Cuál de las siguientes es cierta respecto de esa última pasada con $h = 1$?

Seleccione una:

- ☐ a. La pasada con $h = 1$ es obligatoria pero no es necesario que sea la última.
- ☒ b. Con $h = 1$ el algoritmo se convierte en un ordenamiento por inserción simple, y sólo entonces garantiza que el arreglo quede ordenado. ✓ ¡Correcto!
- ☐ c. No es obligatorio que lo haga, pero favorece un ordenamiento más rápido.
- ☐ d. Con $h = 1$ el algoritmo sólo controla si el arreglo está ya ordenado, y en caso de no estarlo relanza el proceso con otra sucesión de valores h .

¡Correcto!

La respuesta correcta es:

Con $h = 1$ el algoritmo se convierte en un ordenamiento por inserción simple, y sólo entonces garantiza que el arreglo quede ordenado.

Pregunta **10**

Correcta

Se puntúa 1 sobre 1

¿Cuáles de las siguientes son características **correctas** del algoritmo *Heapsort*? (Más de una puede ser cierta... marque TODAS las que considere válidas)

Seleccione una o más de una:

- ☐ a. El algoritmo Heapsort utiliza una cantidad de memoria adicional igual al tamaño del arreglo, para armar el heap o grupo de ordenamiento con el que se ordena el vector.
- ☒ b. El algoritmo Heapsort se basa en encontrar sucesivamente el menor (o el mayor) de entre los elementos que quedan, para llevar ese valor a su casillero final, pero de forma que la búsqueda del menor (o el mayor) en cada vuelta se haga en forma muy veloz. ✓ ¡Correcto!
- ☒ c. El algoritmo Heapsort arma el heap o grupo de ordenamiento con el que se ordena el vector, pero lo hace en el mismo vector, sin usar memoria extra. ✓ ¡Correcto!
- ☒ d. El algoritmo Heapsort es muy eficiente en tiempo de ejecución, tanto para el caso promedio como para el peor caso. ✓ ¡Correcto! Por ahora, no se preocupe del significado profundo de "caso promedio" o "peor caso"...

¡Correcto!

Las respuestas correctas son:

El algoritmo Heapsort es muy eficiente en tiempo de ejecución, tanto para el caso promedio como para el peor caso.,

El algoritmo Heapsort arma el heap o grupo de ordenamiento con el que se ordena el vector, pero lo hace en el mismo vector, sin usar memoria extra.,

El algoritmo Heapsort se basa en encontrar sucesivamente el menor (o el mayor) de entre los elementos que quedan, para llevar ese valor a su casillero final, pero de forma que la búsqueda del menor (o el mayor) en cada vuelta se haga en forma muy veloz.

[◀ Materiales Adicionales para la Ficha 22](#)

Ir a...

Parcial 3 ►

Comenzado el	miércoles, 5 de marzo de 2025, 20:16
Estado	Finalizado
Finalizado en	miércoles, 5 de marzo de 2025, 22:07
Tiempo empleado	1 hora 50 minutos
Puntos	19/20
Calificación	10 de 10 (95%)

Pregunta 1

Correcta

Se puntúa 1 sobre 1

Para cada uno de los algoritmos o procesos listados en la columna de la izquierda, seleccione la expresión de orden que mejor describe su tiempo de ejecución en el peor caso.

Dadas dos matrices de orden $n \times n$, obtener la matriz producto aplicando el método tradicional.

$O(n^3)$ (léase: "orden n al cubo")

✓

Dado un arreglo ya ordenado v con n componentes, buscar un valor x aplicando búsqueda binaria.

$O(\log(n))$

✓

Dado un arreglo v con n componentes, acceder y cambiar el valor del casillero $v[k]$ (con $0 \leq k \leq n-1$).

$O(1)$

✓

Dado un arreglo v con n componentes, ordenarlo de menor a mayor mediante el algoritmo de selección directa.

$O(n^2)$ (léase: "orden n al cuadrado")

✓

Dado un arreglo v con n componentes, buscar un valor x aplicando búsqueda secuencial.

$O(n)$

✓

¡Correcto!

La respuesta correcta es:

Dadas dos matrices de orden $n \times n$, obtener la matriz producto aplicando el método tradicional. → $O(n^3)$ (léase: "orden n al cubo"),

Dado un arreglo ya ordenado v con n componentes, buscar un valor x aplicando búsqueda binaria.
→ $O(\log(n))$,

Dado un arreglo v con n componentes, acceder y cambiar el valor del casillero $v[k]$ (con $0 \leq k \leq n-1$). → $O(1)$,

Dado un arreglo v con n componentes, ordenarlo de menor a mayor mediante el algoritmo de selección directa.
→ $O(n^2)$ (léase: "orden n al cuadrado"),

Dado un arreglo v con n componentes, buscar un valor x aplicando búsqueda secuencial. → $O(n)$

Pregunta 2

Correcta

Se puntúa 1 sobre 1

¿Cuántas comparaciones en el **peor caso** obliga a hacer una búsqueda secuencial en una *lista ordenada* (o en un *arreglo ordenado*) que contenga n valores?

Seleccione una:

- ☐ a. Peor caso: $O(1)$ comparaciones.
- ☐ b. Peor caso: $O(n^2)$ comparaciones.
- ☒ c. **Peor caso: $O(n)$ comparaciones.** ✓ ¡Correcto! Aún estando ordenado el arreglo, en el peor caso una búsqueda secuencial deberá llegar hasta el final si el valor buscado no existe en ese arreglo (o existe y está muy atrás)
- ☐ d. Peor caso: $O(\log(n))$ comparaciones.

¡Correcto!

La respuesta correcta es:

Peor caso: $O(n)$ comparaciones.

Pregunta 3

Correcta

Se puntúa 1 sobre 1

¿Cuál es la diferencia entre el **peor caso** y el **caso promedio** en el análisis de algoritmos?

Seleccione una:

- ☒ a. **El **peor caso** es la configuración de datos de entrada más desfavorable para el algoritmo, mientras que el **caso promedio** describe una configuración aleatoria de datos (no pensada ni para favorecer ni para desfavorecer al algoritmo).** ✓ ¡Correcto!
- ☐ b. El **peor caso** es la configuración de datos de entrada más favorable para el algoritmo, mientras que el **caso promedio** describe una configuración aleatoria de datos (no pensada ni para favorecer ni para desfavorecer al algoritmo)
- ☐ c. El **peor caso** es la configuración de datos de entrada más desfavorable para el algoritmo, mientras que el **caso promedio** describe una configuración de datos pensada para favorecer al algoritmo.
- ☐ d. El **peor caso** es la configuración de datos de entrada más favorable para el algoritmo, mientras que el **caso promedio** describe una configuración de datos pensada para desfavorecer al algoritmo.

¡Correcto!

La respuesta correcta es:

El **peor caso** es la configuración de datos de entrada más desfavorable para el algoritmo, mientras que el **caso promedio** describe una configuración aleatoria de datos (no pensada ni para favorecer ni para desfavorecer al algoritmo).

Pregunta 4

Correcta

Se puntúa 1 sobre 1

¿Cuáles de los siguientes son *factores de eficiencia comunes* a considerar en el análisis de algoritmos? (Más de una respuesta puede ser válida... marque *todas* las que considere correctas).

Seleccione una o más de una:

- ☐ a. La calidad aparente de la interfaz de usuario.
- ☒ b. La complejidad aparente del código fuente. ✓ ¡Correcto!
- ☒ c. El consumo de memoria. ✓ ¡Correcto!
- ☒ d. El tiempo de ejecución. ✓ ¡Correcto!

¡Correcto!

Las respuestas correctas son:

El tiempo de ejecución.,

El consumo de memoria.,

La complejidad aparente del código fuente.

Pregunta 5

Incorrecta

Se puntúa 0 sobre 1

¿Qué significa decir que un algoritmo dado tiene un tiempo de ejecución $O(1)$?

Seleccione una:

- ☐ a. El tiempo de ejecución es lineal: si aumenta el número de datos, aumenta el tiempo en la misma proporción.
- ☒ b. El tiempo de ejecución siempre es de un segundo, sin importar la cantidad de datos. ✗ Incorrecto... aunque es cierto que la cantidad de datos no importará...
- ☐ c. El tiempo de ejecución es constante, sin importar la cantidad de datos.
- ☐ d. El tiempo de ejecución es logarítmico: a medida que aumenta el número de datos, aumenta el tiempo pero en forma muy suave.

La respuesta correcta es:

El tiempo de ejecución es constante, sin importar la cantidad de datos.

Pregunta 6

Correcta

Se puntúa 1 sobre 1

¿Qué significa decir que un algoritmo dado tiene un tiempo de ejecución $O(n^2)$?

Seleccione una:

- ☐ a. A medida que aumenta el número de datos, aumenta el tiempo pero en forma muy suave: el conjunto de datos se divide en dos. se procesa una de las mitades, se desecha la otra y se repite el proceso hasta que no pueda volver a dividirse la mitad que haya quedado.
- ☒ b. El proceso normalmente consiste en dos ciclos (uno dentro del otro) de aproximadamente n iteraciones cada uno, de forma que las operaciones críticas se aplican un número cuadrático de veces. ✓ ¡Correcto!
- ☐ c. El tiempo de ejecución es constante, sin importar la cantidad de datos.
- ☐ d. El tiempo de ejecución es lineal: si aumenta el número de datos, aumenta el tiempo en la misma proporción.

¡Correcto!

La respuesta correcta es:

El proceso normalmente consiste en dos ciclos (uno dentro del otro) de aproximadamente n iteraciones cada uno, de forma que las operaciones críticas se aplican un número cuadrático de veces.

Pregunta 7

Correcta

Se puntúa 1 sobre 1

¿Qué significa decir que un algoritmo dado tiene un tiempo de ejecución $O(n)$?

Seleccione una:

- ☒ a. El tiempo de ejecución es lineal: si aumenta el número de datos, aumenta el tiempo en la misma proporción. ✓ ¡Correcto!
- ☐ b. El proceso normalmente consiste en dos ciclos (uno dentro del otro) de aproximadamente n iteraciones cada uno, de forma que las operaciones críticas se aplican un número cuadrático de veces.
- ☐ c. El tiempo de ejecución es constante, sin importar la cantidad de datos.
- ☐ d. A medida que aumenta el número de datos, aumenta el tiempo pero en forma muy suave: el conjunto de datos se divide en dos. se procesa una de las mitades, se desecha la otra y se repite el proceso hasta que no pueda volver a dividirse la mitad que haya quedado.

¡Correcto!

La respuesta correcta es:

El tiempo de ejecución es lineal: si aumenta el número de datos, aumenta el tiempo en la misma proporción.

Pregunta 8

Correcta

Se puntúa 1 sobre 1

¿Qué significa decir que un algoritmo dado tiene un tiempo de ejecución $O(\log(n))$?

Seleccione una:

- ☐ a. El tiempo de ejecución es lineal: si aumenta el número de datos, aumenta el tiempo en la misma proporción.
- ☒ b. A medida que aumenta el número de datos, aumenta el tiempo pero en forma muy suave: el conjunto de datos se divide en dos. se procesa una de las mitades, se desecha la otra y se repite el proceso hasta que no pueda volver a dividirse la mitad que haya quedado. ✓ ¡Correcto!
- ☐ c. El proceso normalmente consiste en dos ciclos (uno dentro del otro) de aproximadamente n iteraciones cada uno, de forma que las operaciones críticas se aplican un número cuadrático de veces.
- ☐ d. El tiempo de ejecución es constante, sin importar la cantidad de datos.

¡Correcto!

La respuesta correcta es:

A medida que aumenta el número de datos, aumenta el tiempo pero en forma muy suave: el conjunto de datos se divide en dos. se procesa una de las mitades, se desecha la otra y se repite el proceso hasta que no pueda volver a dividirse la mitad que haya quedado.

Pregunta 9

Correcta

Se puntúa 1 sobre 1

Suponga que dispone de cuatro algoritmos diferentes para resolver el mismo problema, y que se sabe que los tiempos de ejecución (en el peor caso) son, respectivamente: $O(n \cdot \log(n))$, $O(n^2)$, $O(n^3)$ y $O(n)$.

¿Cuál de esos tres algoritmos debería elegir, suponiendo que todos hacen el mismo consumo razonable de memoria?

Seleccione una:

- ☐ a. El algoritmo cuyo tiempo de ejecución es $O(n^3)$
- ☒ b. El algoritmo cuyo tiempo de ejecución es $O(n)$ ✓ ¡Correcto! Efectivamente, este algoritmo sería el más rápido...
- ☐ c. El algoritmo cuyo tiempo de ejecución es $O(n \cdot \log(n))$
- ☐ d. El algoritmo cuyo tiempo de ejecución es $O(n^2)$

¡Correcto!

La respuesta correcta es:

El algoritmo cuyo tiempo de ejecución es $O(n)$

Pregunta **10**

Correcta

Se puntúa 1 sobre 1

¿Con qué nombre general se conoce en la *Teoría de la Complejidad* a un problema para el cual sólo se conocen algoritmos cuyo tiempo de ejecución es exponencial (o sea, problemas para los que todas las soluciones conocidas son algoritmos con tiempo $O(2^n)$)?

Seleccione una:

- ☐ a. Problemas Imperdonables
- ☐ b. Problemas Inmanejables
- ☐ c. Problemas Irresolubles
- ☒ d. Problemas Intratables ✓ ¡Correcto!

¡Correcto!

La respuesta correcta es:

Problemas Intratables

Pregunta 11

Correcta

Se puntúa 1 sobre 1

En la columna de la izquierda se muestra el código fuente en Python de diversos procesos sencillos. Para cada uno de ellos, seleccione de la lista de la derecha la expresión en notación *Big O* que mejor describa el tiempo de ejecución de cada proceso para el peor caso. (Aclaración: una expresión como n^2 debe entenderse como "*n al cuadrado*" o n^2).

```
n = int(input('N: '))
ac = 0
for i in range(n):
    for j in range(i+1, n):
        ac += i*j
print(ac)
```

 $O(n^2)$ 

```
n = int(input('N: '))
ac = 0
for i in range(n):
    for j in range(i+1, n):
        ac += i*j

for i in range(n):
    for j in range(n):
        for k in range(n):
            ac += (i+j+k)
print(ac)
```

 $O(n^3)$ 

```
n = int(input('N: '))
ac = 0
for i in range(n):
    ac += i
print(ac)
```

 $O(n)$ 

```
ac = 0
for i in range(10):
    ac += i
print(ac)
```

 $O(1)$ 

¡Correcto!

La respuesta correcta es:

```
n = int(input('N: '))
ac = 0
for i in range(n):
    for j in range(i+1, n):
        ac += i*j
print(ac)
```

→ $O(n^2)$,

```
n = int(input('N: '))
ac = 0
for i in range(n):
    for j in range(i+1, n):
        ac += i*j

for i in range(n):
    for j in range(n):
        for k in range(n):
            ac += (i+j+k)
print(ac)
```

→ $O(n^3)$,

```
n = int(input('N: '))
ac = 0
for i in range(n):
    ac += i
print(ac)
```

→ $O(n)$,

```
ac = 0
for i in range(10):
    ac += i
print(ac)
```

→ $O(1)$ Pregunta **12**

Correcta

Se puntúa 1 sobre 1

Analice el siguiente esquema de una función en Python:

```
def procesar(n, m):
    for i in range(n+1):
        for j in range(m+1):
            # ... acciones sencillas a realizar...
            # ... suponga que no hay otro ciclo aquí...
            # ... y que sólo aparecen operaciones de tiempo constante...
```

¿Cuál de las siguientes expresiones de orden describe mejor el tiempo de ejecución de esta función en el peor caso?

Seleccione una:

- ☐ a. $O(n*n)$
- ☒ b. $O(n*m)$ ✓ ¡Correcto! Efectivamente, no hay problema alguno en que una expresión de orden se base en dos o más variables si el tamaño del problema depende de esas variables.
- ☐ c. $O(n)$
- ☐ d. $O(m)$

¡Correcto!

La respuesta correcta es:

 $O(n*m)$

Pregunta 13

Correcta

Se puntúa 1 sobre 1

Si se realiza un análisis preciso del ordenamiento por *Selección Directa* para un arreglo de n componentes, se llega a la conclusión que ese algoritmo hará $n-1$ pasadas, con $n-1$ comparaciones en la primera, $n-2$ en la segunda, y así sucesivamente reduciendo de a 1 la cantidad de comparaciones hasta hacer sólo una comparación en la última pasada. Por lo tanto, el algoritmo hará *invariablemente* una cantidad total de $\frac{1}{2}(n^2 - n)$ comparaciones. Sabiendo esto, ¿cuáles de las siguientes expresiones son correctas para describir la cantidad de comparaciones que hará el algoritmo, usando distintos tipos de notaciones? (Más de una respuesta puede ser correcta. Marque TODAS las que considere correctas)

Seleccione una o más de una:

- ☒ a. Cantidad de comparaciones: $\Theta(n^2)$ ✓ ¡Correcto!
- ☐ b. Cantidad de comparaciones: $o(n^2)$
- ☒ c. Cantidad de comparaciones: $\Omega(n^2)$ ✓ ¡Correcto!
- ☒ d. Cantidad de comparaciones: $O(n^2)$ ✓ ¡Correcto!

¡Correcto!

Las respuestas correctas son:

Cantidad de comparaciones: $O(n^2)$,Cantidad de comparaciones: $\Omega(n^2)$,Cantidad de comparaciones: $\Theta(n^2)$

Pregunta 14

Correcta

Se puntúa 1 sobre 1

¿Cuál es la **principal** característica de todos los métodos de ordenamiento conocidos como **métodos simples** o **directos**?

Seleccione una:

- ☐ a. Son muy fáciles de programar.
- ☐ b. Son muy veloces para cualquier tamaño del arreglo a ordenar.
- ☒ c. Tienen un tiempo de ejecución de orden cuadrático en el peor caso. ✓ ¡Correcto!
- ☐ d. Tienen un tiempo de ejecución de orden lineal en el peor caso.

¡Correcto!

La respuesta correcta es:

Tienen un tiempo de ejecución de orden cuadrático en el peor caso.

Pregunta 15

Correcta

Se puntúa 1 sobre 1

¿Cuáles de las siguientes son características **correctas** del algoritmo *Shellsort*? (Más de una puede ser cierta... marque TODAS las que considere válidas)

Seleccione una o más de una:

- ☐ a. El algoritmo *Shellsort* consiste en una mejora del algoritmo de *Selección Directa*, consistente en buscar iterativamente el menor (o el mayor) entre los elementos que quedan en el vector, para llevarlo a su posición correcta, pero de forma que la búsqueda del menor en cada vuelta se haga en tiempo logarítmico.
- ☐ b. En el caso promedio, el algoritmo *Shellsort* es tan eficiente como el *Heapsort* o el *Quicksort*, con tiempo de ejecución $O(n \log(n))$.
- ☒ c. El algoritmo *Shellsort* es complejo de analizar para determinar su rendimientos en forma matemática. Se sabe que para la serie de incrementos decrecientes usada en la implementación vista en las clases de la asignatura, tiene un tiempo de ejecución para el peor caso de $O(n^{1.5})$. ✓ ¡Correcto!
- ☒ d. El algoritmo *Shellsort* consiste en una mejora del algoritmo de *Inserción Directa* (o *Inserción Simple*), consistente en armar suconjuntos ordenados con elementos a distancia $h > 1$ en las primeras fases, y terminar con $h = 1$ en la última. ✓ ¡Correcto!

¡Correcto!

Las respuestas correctas son:

El algoritmo *Shellsort* consiste en una mejora del algoritmo de *Inserción Directa* (o *Inserción Simple*), consistente en armar suconjuntos ordenados con elementos a distancia $h > 1$ en las primeras fases, y terminar con $h = 1$ en la última.,

El algoritmo *Shellsort* es complejo de analizar para determinar su rendimientos en forma matemática. Se sabe que para la serie de incrementos decrecientes usada en la implementación vista en las clases de la asignatura, tiene un tiempo de ejecución para el peor caso de $O(n^{1.5})$.

Pregunta 16

Correcta

Se puntúa 1 sobre 1

¿Qué diferencia existe entre el **conteo exhaustivo** de operaciones críticas y el **análisis asintótico** del comportamiento de una función en el análisis de algoritmos?

Seleccione una:

- ☐ a. Ninguna. Ambas se refieren a la misma técnica básica del análisis de algoritmos
- ☒ b. El **conteo exhaustivo** busca determinar una expresión o fórmula que exprese de manera rigurosa la cantidad de operaciones críticas que lleva a cabo un algoritmo, mientras que el **análisis asintótico** busca determinar el comportamiento general de una función para valores muy grandes del tamaño del problema. ✓ ¡Correcto!
- ☐ c. El **análisis asintótico** busca determinar una expresión o fórmula que exprese de manera rigurosa la cantidad de operaciones críticas que lleva a cabo un algoritmo, mientras que el **conteo exhaustivo** busca determinar el comportamiento general de una función para valores muy grandes del tamaño del problema.
- ☐ d. El **conteo exhaustivo** busca determinar una expresión o fórmula que exprese de manera rigurosa la cantidad de operaciones **críticas** que lleva a cabo un algoritmo, mientras que el **análisis asintótico** busca determinar una expresión o fórmula que exprese de manera rigurosa la cantidad de operaciones **no críticas** que lleva a cabo un algoritmo.

¡Correcto!

La respuesta correcta es:

El **conteo exhaustivo** busca determinar una expresión o fórmula que exprese de manera rigurosa la cantidad de operaciones críticas que lleva a cabo un algoritmo, mientras que el **análisis asintótico** busca determinar el comportamiento general de una función para valores muy grandes del tamaño del problema.

Pregunta **17**

Correcta

Se puntúa 1 sobre 1

¿Qué se entiende, en el contexto del Análisis de Algoritmos, por un *Orden de Complejidad*?

Seleccione una:

- ☒ a. Un conjunto o familia de funciones matemáticas que se comportan asintóticamente de la misma forma. ✓ ¡Correcto!
- ☐ b. Un conjunto o familia de algoritmos que resuelven el mismo problema.
- ☐ c. Un conjunto o familia de subrutinas con similares objetivos (equivalente al concepto de *módulo*).
- ☐ d. Un conjunto de datos ordenados.

¡Correcto!

Las respuestas correctas son:

Un conjunto o familia de funciones matemáticas que se comportan asintóticamente de la misma forma.,

Un conjunto o familia de algoritmos que resuelven el mismo problema.

Pregunta **18**

Correcta

Se puntúa 1 sobre 1

Si los algoritmos de *ordenamiento simples* tienen todos un tiempo de ejecución $O(n^2)$ en el peor caso, entonces: ¿cómo explica que las mediciones efectivas de los tiempos de ejecución de cada uno sean diferentes frente al mismo arreglo?

Seleccione una:

- ☒ a. La notación *Big O* rescata el término más significativo en la expresión que calcula el rendimiento, descartando constantes y otros términos que podrían no coincidir en los tres algoritmos. ✓ ¡Correcto!
- ☐ b. Los tiempos deben coincidir. Si hay diferencias, se debe a errores en los instrumentos de medición o a un planteo incorrecto del proceso de medición.
- ☐ c. La notación *Big O* no se debe usar para estimar el comportamiento en el peor caso, sino sólo para el caso medio.
- ☐ d. La notación *Big O* no se usa para medir tiempos sino para contar comparaciones u otro elemento de interés. Es un error, entonces, decir que los tiempos tienen "orden n cuadrado".

¡Correcto!

La respuesta correcta es:

La notación *Big O* rescata el término más significativo en la expresión que calcula el rendimiento, descartando constantes y otros términos que podrían no coincidir en los tres algoritmos.

Pregunta 19

Correcta

Se puntúa 1 sobre 1

Se tiene un algoritmo que realiza cierta cantidad de procesos sobre un conjunto de n datos y un minucioso análisis matemático ha determinado que la cantidad de procesos que el algoritmo realiza en el peor caso viene descrito por la función $f(n) = 3n^3 + 5n^2 + 2n^{1.5}$. ¿Cuál de las siguientes expresiones representa mejor el orden del algoritmo para el peor caso?

Seleccione una:

- ☐ a. $O(3n^3 + 5n^2 + 2n^{1.5})$
- ☐ b. $O(n^{1.5})$
- ☐ c. $O(n^3 + n^2)$
- ☒ d. $O(n^3)$ ✓ ¡Correcto!

¡Correcto!

La respuesta correcta es:

 $O(n^3)$

Pregunta 20

Correcta

Se puntúa 1 sobre 1

¿Cuáles de las siguientes *son correctas* en cuanto a los tiempos de ejecución de los algoritmos de ordenamiento clásicos? (Más de una puede ser cierta... marque TODAS las que considere válidas)

Seleccione una o más de una:

- ☐ a. Algoritmo Shell Sort: $O(n^2)$ en el peor caso para la serie de incrementos decrecientes vista en clase.
- ☒ b. Algoritmo Heap Sort: $O(n \log(n))$ tanto para el caso promedio como para el peor caso. ✓ ¡Correcto!
- ☒ c. Algoritmo Quick Sort: $O(n \log(n))$ en el caso promedio, pero $O(n^2)$ en el peor caso. ✓ ¡Correcto!
- ☒ d. Algoritmos directos o simples: $O(n^2)$ en el peor caso para todos ellos. ✓ ¡Correcto!

¡Correcto!

Las respuestas correctas son:

Algoritmos directos o simples: $O(n^2)$ en el peor caso para todos ellos.,Algoritmo Quick Sort: $O(n \log(n))$ en el caso promedio, pero $O(n^2)$ en el peor caso.,Algoritmo Heap Sort: $O(n \log(n))$ tanto para el caso promedio como para el peor caso.[◀ Ficha 27 - Videos Explicativos \(Adicionales\)](#)

Ir a...

Parcial 4 ►

[Página Principal](#) / [Mis cursos](#) / [AED-2024-IntLib-\(Feb2025\)](#) / [Ficha 28](#) / [Cuestionario 28 \[Temas: hasta Ficha 28\]](#)

Comenzado el	miércoles, 5 de marzo de 2025, 22:08
Estado	Finalizado
Finalizado en	miércoles, 5 de marzo de 2025, 22:34
Tiempo empleado	26 minutos 22 segundos
Puntos	13/13
Calificación	10 de 10 (100%)

Pregunta **1**

Correcta

Se puntúa 1 sobre 1

¿Cuál es la diferencia entre la *abstracción de datos* y la *abstracción funcional*?

Seleccione una:

- ☐ a. No existe un mecanismo de abstracción de datos ni un mecanismo de abstracción funcional. Existe sólo un mecanismo de abstracción, sin dividir en abstracción de datos y abstracción funcional.
- ☒ b. La abstracción de datos busca captar el conjunto de datos más relevante para representar un tipo abstracto, ✓ ¡Correcto! mientras que la funcional buscar determinar el conjunto de procesos relevante para esos datos.
- ☐ c. Ninguna. Son sólo dos formas de referirse al mecanismo de abstracción.
- ☐ d. La abstracción de datos busca captar el conjunto de procesos relevante para el tipo abstracto que se quiere implementar, mientras que la funcional buscar determinar el conjunto de datos más relevante para implementar ese tipo.

¡Correcto!

La respuesta correcta es:

La abstracción de datos busca captar el conjunto de datos más relevante para representar un tipo abstracto, mientras que la funcional buscar determinar el conjunto de procesos relevante para esos datos.

Pregunta 2

Correcta

Se puntúa 1 sobre 1

Suponga que se quiere implementar un nuevo tipo de datos abstracto llamado *Fecha*, para permitir la manipulación de fechas en un programa en Python para gestión de eventos sociales (casamientos, fiestas de cumpleaños, reuniones de egresados, etc.) ¿Cuál de las siguientes estrategias de abstracción sería la más adecuada?

Seleccione una:

- ☐ a. *Abstracción de datos*: una tupla con tres componentes para el año, el mes y el día. *Abstracción funcional*: funciones para buscar una fecha dada, reservar una fecha para un evento, liberar una fecha para un evento y modificar los datos de una fecha para un evento.
- ☐ b. *Abstracción de datos*: un registro con campos para el año, el mes, el día, el nombre del cliente, el tipo de evento, el monto presupuestado y una proyección meteorológica para ese día. *Abstracción funcional*: funciones para buscar una fecha dada, reservar una fecha para un evento, liberar una fecha para un evento y modificar los datos de una fecha para un evento.
- ☒ c. *Abstracción de datos*: un registro con tres campos para el año, el mes y el día. *Abstracción funcional*: funciones para buscar una fecha dada, reservar una fecha para un evento, liberar una fecha para un evento y modificar los datos de una fecha para un evento. ✓ ¡Correcto!
- ☐ d. *Abstracción de datos*: una variable de tipo *cadena de caracteres* para representar toda la fecha (por ejemplo: '2015/08/31'). *Abstracción funcional*: funciones para mostrar la fecha en distintos colores, imprimir la fecha en forma de cartel anunciador, cambiar el número del mes por el nombre del mes.

¡Correcto!

La respuesta correcta es:

Abstracción de datos: un registro con tres campos para el año, el mes y el día. *Abstracción funcional*: funciones para buscar una fecha dada, reservar una fecha para un evento, liberar una fecha para un evento y modificar los datos de una fecha para un evento.

Pregunta 3

Correcta

Se puntúa 1 sobre 1

¿Cuál de las siguientes expresiones describe la forma de trabajo general de una *Pila*?

Seleccione una:

- ☒ a. LIFO (Last In - First Out) ✓ ¡Correcto!
- ☐ b. FIFO (First In - First Out)
- ☐ c. OIRO (Ordered In - Random Out)
- ☐ d. OIFO (Ordered In - First Out)

¡Correcto!

La respuesta correcta es:

LIFO (Last In - First Out)

Pregunta 4

Correcta

Se puntúa 1 sobre 1

¿Cuál de las siguientes situaciones de programación es la principal aplicación de una **Pila?**

Seleccione una:

- ☒ a. **Procesar una secuencia de datos en orden inverso al de su entrada.** ✓ ¡Correcto!
- ☐ b. Procesar una secuencia de datos en orden aleatorio.
- ☐ c. Procesar una secuencia de datos en orden de menor a mayor.
- ☐ d. Procesar una secuencia de datos en el mismo orden en el que ingresaron.

¡Correcto!

La respuesta correcta es:

Procesar una secuencia de datos en orden inverso al de su entrada.

Pregunta 5

Correcta

Se puntúa 2 sobre 2

Suponga que se tiene una pila p con capacidad para almacenar números enteros y que las operaciones básicas $pop()$, $peek()$ y $push()$ están correctamente implementadas en el módulo *stack.py* presentado en clases. Suponga que se han insertado los siguientes valores: [8 - 6 - 5 - 3 - 7] (el número 8 es el valor del frente o tope de la Pila). ¿Cuál de las siguientes secuencias de instrucciones permite retirar el valor 5 de la pila, pero dejando el 6 y 8 nuevamente arriba? (es decir: ¿cuál de las siguientes secuencias, dejaría la pila p en el estado [8 - 6 - 3 - 7]?)

Seleccione una:

- ☐ a. `n1 = p.pop()`
`n2 = p.pop()`
`x = p.pop()`
- ☐ b. `n1 = p.pop()`
`n2 = p.pop()`
`x = p.peak()`
`p.push(n2)`
`p.push(n1)`
- ☒ c. `n1 = p.pop()` ✓ ¡Correcto!
`n2 = p.pop()`
`x = p.pop()`
`p.push(n2)`
`p.push(n1)`
- ☐ d. `n1 = p.pop()`
`n2 = p.pop()`
`x = p.pop()`
`p.push(n1)`
`p.push(n2)`

¡Correcto!

La respuesta correcta es:

```
n1 = p.pop()
n2 = p.pop()
x = p.pop()
p.push(n2)
p.push(n1)
```


Pregunta 6

Correcta

Se puntúa 2 sobre 2

Suponga que se tiene una pila p en la cual se almacenaron ya una cierta cantidad de datos. ¿Cuál de las siguientes secuencias de instrucciones permite *invertir* la pila? (o sea: si la pila original p era: [3 - 4 - 6 - 7] (con el 3 al frente) ¿cuál de las siguientes permitiría dejar la pila p en el estado [7 - 6 - 4 - 3] (con el 7 al frente)?) (Suponga que $p2$ y $p3$ también son pilas, inicialmente vacías y listas para usar)

Seleccione una:

- ☐ a.

```
while not p.is_empty():
    p2.push(p.pop())
while not p2.is_empty():
    p3.push(p2.pop())
while not p3.is_empty():
    p2.push(p3.pop())
```
- ☒ b.

```
while not p.is_empty():
    p2.push(p.pop())
while not p2.is_empty():
    p3.push(p2.pop())
while not p3.is_empty():
    p.push(p3.pop())
```

 ✓ ¡Correcto!
- ☐ c.

```
while not p.is_empty():
    p2.push(p.pop())
while not p2.is_empty():
    p3.push(p2.pop())
```
- ☐ d.

```
while not p.is_empty():
    p2.push(p.pop())
while not p2.is_empty():
    p.push(p2.pop())
```

¡Correcto!

La respuesta correcta es:

```
while not p.is_empty():
    p2.push(p.pop())
while not p2.is_empty():
    p3.push(p2.pop())
while not p3.is_empty():
    p.push(p3.pop())
```

Pregunta **7**

Correcta

Se puntúa 1 sobre 1

¿Cuál de las siguientes expresiones describe la forma de trabajo general de una **Cola**?

Seleccione una:

- ☐ a. LIFO (Last In - First Out)
- ☐ b. OIRO (Ordered In - Random Out)
- ☐ c. OIFO (Ordered In - First Out)
- ☒ d. **FIFO (First In - First Out) ✓** ¡Correcto!

¡Correcto!

La respuesta correcta es:

FIFO (First In - First Out)

Pregunta **8**

Correcta

Se puntúa 1 sobre 1

¿Cuál de las siguientes situaciones de programación es la principal aplicación de una **Cola**?

Seleccione una:

- ☐ a. Procesar una secuencia de datos en orden de menor a mayor.
- ☐ b. Procesar una secuencia de datos en orden inverso al de su entrada.
- ☒ c. **Procesar una secuencia de datos en el mismo orden en el que ingresaron. ✓** ¡Correcto!
- ☐ d. Procesar una secuencia de datos en orden aleatorio.

¡Correcto!

La respuesta correcta es:

Procesar una secuencia de datos en el mismo orden en el que ingresaron.

Pregunta 9

Correcta

Se puntúa 2 sobre 2

Suponga que se tiene una cola c en la cual se almacenaron ya una cierta cantidad de datos. ¿Cuál de las siguientes secuencias de instrucciones permite *invertir* la cola? (o sea: si la cola original c era: [3 - 4 - 6 - 7] (con el 3 al frente), ¿cuál de las siguientes permitiría dejar la cola c en el estado [7 - 6 - 4 - 3] (con el 7 al frente)?)

Seleccione una:

- ☐ a.

```
# suponga que p1 y p1 son dos PILAS inicialmente vacías, y listas para usar
while not c.is_empty():
    p1.push(c.remove())
while not p1.is_empty():
    p2.push(p1.pop())
while not p2.is_empty():
    c.add(p2.pop())
```
- ☐ b.

```
# suponga que c2 y c3 son dos colas inicialmente vacías, y listas para usar
while not c.is_empty():
    c2.add(c2, c.remove())
while not c2.is_empty():
    c2.add(c2.remove())
while not c3.is_empty():
    c.add(c3.remove())
```
- ☐ c.

```
# suponga que c2 es OTRA cola vacía y lista para usar...
while not c.is_empty():
    c2.add(c.remove())
while not c2.is_empty():
    c.add(c2.remove())
```
- ☒ d.

```
# suponga que p es una PILA vacía y lista para usar...
while not c.is_empty():
    p.push(c.remove())
while not p.is_empty():
    c.add(p.pop())
```

¡Correcto! Efectivamente... si desea la inversión de una secuencia, una pila le permitirá lograrlo...

¡Correcto!

La respuesta correcta es:

```
# suponga que p es una PILA vacía y lista para usar...
while not c.is_empty():
    p.push(c.remove())
while not p.is_empty():
    c.add(p.pop())
```

Pregunta **10**

Correcta

Se puntúa 1 sobre 1

Una *Cola de Prioridad* es una cola en la cual los elementos se insertan en algún orden, pero tal que cuando se pide retirar un elemento se obtiene siempre *el menor* de los valores almacenados en la cola. ¿Cuál de las siguientes estrategias podría ser una forma básica de implementar una *Cola de Prioridad* en las condiciones aquí expresadas?

Seleccione una:

- ☐ a. Soporte: una variable de tipo *list* en Python. Inserción: siempre al frente. Eliminación: siempre el primer elemento.
- ☐ b. Soporte: una variable de tipo *list* en Python. Inserción: *ordenada de mayor a menor* (mantener el arreglo siempre ordenado de mayor a menor: para insertar un nuevo valor *x*, recorrer el arreglo y al encontrar el primer menor, a *x* detener el ciclo y añadir allí a *x* con un corte de índices. Eliminación: siempre el primer elemento.
- ☒ c. Soporte: una variable de tipo *list* en Python. Inserción: *ordenada de menor a mayor* (mantener el arreglo siempre ordenado de menor a mayor: para insertar un valor *x*, recorrer el arreglo y al encontrar el primer mayor a *x* detener el ciclo y añadir allí a *x* con un corte de índices. Eliminación: siempre el primer elemento. ✓
- ☐ d. Soporte: una variable de tipo *list* en Python. Inserción: siempre al final. Eliminación: siempre el último elemento.

¡Correcto! Aunque entienda que si se hace así, logrará el objetivo pero la inserción tendrá un tiempo de ejecución $O(n)$ mientras que la eliminación será de tiempo constante ($O(1)$).

¡Correcto!

La respuesta correcta es:

Soporte: una variable de tipo *list* en Python. Inserción: *ordenada de menor a mayor* (mantener el arreglo siempre ordenado de menor a mayor: para insertar un valor *x*, recorrer el arreglo y al encontrar el primer mayor a *x* detener el ciclo y añadir allí a *x* con un corte de índices. Eliminación: siempre el primer elemento.

[◀ Materiales Adicionales para la Ficha 28](#)

Ir a...

[Guía 28 de Ejercicios Prácticos ►](#)

[Página Principal](#) / [Mis cursos](#) / [AED-2024-IntLib-\(Feb2025\)](#) / [Ficha 29](#) / [Cuestionario 29 \[Temas: hasta Ficha 29\]](#)

Comenzado el	miércoles, 5 de marzo de 2025, 22:38
Estado	Finalizado
Finalizado en	miércoles, 5 de marzo de 2025, 22:56
Tiempo empleado	18 minutos 5 segundos
Calificación	10 de 10 (100%)

Pregunta **1**

Correcta

Se puntúa 1 sobre 1

Sabemos que en un árbol binario (sea o no de búsqueda) un nodo puede tener a lo sumo dos hijos. Por otra parte, un nodo tendrá un único nodo padre (salvo la raíz, que no tiene padre) ¿Qué parte de la definición de árbol binario es la que impide que un nodo pueda tener más de un padre?

Seleccione una:

- ☒ a. La sección en la que se afirma que los dos subconjuntos que dependen del nodo raíz son conjuntos separados o mutuamente excluyentes. ✓ ¡Correcto!
- ☐ b. La sección en la que se afirma que el árbol puede estar vacío o puede contener un nodo designado como la raíz del árbol.
- ☐ c. La sección en la que se afirma que del nodo raíz se desprenden dos subconjuntos.
- ☐ d. La sección en la que se afirma que los dos subconjuntos que dependen del nodo raíz pueden estar vacíos.

¡Correcto!

La respuesta correcta es:

La sección en la que se afirma que los dos subconjuntos que dependen del nodo raíz son conjuntos separados o mutuamente excluyentes.

Pregunta **2**

Correcta

Se puntúa 1 sobre 1

¿Cuál de las siguientes es CIERTA respecto de los árboles binarios de búsqueda?

Seleccione una:

- ☐ a. Siempre tienen altura mínima.
- ☒ b. Si el árbol tiene n nodos, el tiempo de búsqueda en el peor caso es $O(\log(n))$ si la altura del árbol h es $O(\log(n))$. ✓ ¡Correcto!
- ☐ c. Si el árbol tiene n nodos, el tiempo de búsqueda en el peor caso es $O(n \cdot \log(n))$ si la altura del árbol h es $O(\log(n))$.
- ☐ d. Siempre tienen altura máxima.

¡Correcto!

La respuesta correcta es:

Si el árbol tiene n nodos, el tiempo de búsqueda en el peor caso es $O(\log(n))$ si la altura del árbol h es $O(\log(n))$.

Pregunta 3

Correcta

Se puntúa 1 sobre 1

¿Cuántas comparaciones **por nivel** se realizan en un árbol de búsqueda de n nodos cuando se busca una clave? (no importa si la búsqueda llega o no al último nivel del árbol: sólo queremos saber cuántas comparaciones se hacen en aquellos niveles hasta los que llegue la búsqueda)

Seleccione una:

- ☒ a. Comparaciones por nivel: 1 (una) ✓ ¡Correcto!
- ☐ b. Comparaciones por nivel: n
- ☐ c. Comparaciones por nivel: n^2
- ☐ d. Comparaciones por nivel: $\log(n)$

¡Correcto!

La respuesta correcta es: Comparaciones por nivel: 1 (una)

Pregunta 4

Correcta

Se puntúa 1 sobre 1

Considere el método `contains()` que busca un objeto x en un árbol de búsqueda de n nodos, según se vió en las fichas de clase de la asignatura:

```
def contains(self, x):  
    if x is None:  
        return False  
  
    p = self.root  
    while p is not None:  
        if x == p.info:  
            return True  
  
        elif x < p.info:  
            p = p.left  
        else:  
            p = p.right  
  
    return False
```

Si el algoritmo de inserción mediante el cual se ha creado el árbol es el algoritmo simple visto en la ficha y en clases, sin control de altura o balance, ¿cuáles de las siguientes afirmaciones son correctas en relación al algoritmo implementado en el método `contains()`? (Más de una respuesta puede ser cierta. Marque todas las que considere correctas).

Seleccione una o más de una:

- ☒ a. El tiempo de ejecución del algoritmo mostrado es $O(\log(n))$ en el peor caso, pero sólo si el árbol tiene altura $h = O(\log(n)) = \text{mínima}$. ✓ ¡Correcto!
- ☐ b. El tiempo de ejecución del algoritmo mostrado es $O(1)$ en el peor caso.
- ☐ c. El tiempo de ejecución del algoritmo mostrado es $O(\log(n))$ en el peor caso.
- ☒ d. El tiempo de ejecución del algoritmo mostrado es $O(n)$ en el peor caso. ✓ ¡Correcto! El peor caso se daría con un árbol convertido es una diagonal o en un zig zag de altura n , que con el algoritmo de inserción ingenuo podría ocurrir...

¡Correcto!

Las respuestas correctas son: El tiempo de ejecución del algoritmo mostrado es $O(n)$ en el peor caso.,

El tiempo de ejecución del algoritmo mostrado es $O(\log(n))$ en el peor caso, pero sólo si el árbol tiene altura $h = O(\log(n)) = \text{mínima}$.

Pregunta 5

Correcta

Se puntúa 1 sobre 1

Suponga que desconoce la forma gráfica de un árbol binario (**que no es necesariamente de búsqueda**) con el que está trabajando, pero se le da la posibilidad de conocer una o dos secuencias de recorrido de ese árbol, en distintos órdenes. *¿Con cuáles de las siguientes secuencias de recorrido se garantiza que podrá volver a dibujar el árbol original, sin ambigüedades ni errores?* (Piense un poco para responder, y en todo caso, haga algunos intentos dibujando un árbol y tratando de reconstruirlo a partir de sus secuencias de recorrido) **(Observación: más de una respuesta puede ser correcta. Marque todas las que considere válidas). Recuerde: se supone que el árbol original es binario, pero NO NECESARIAMENTE de búsqueda.**

Seleccione una o más de una:

- ☒ a. Podrá volver a dibujar el árbol si conoce la secuencia de recorrido en *postorden* y la secuencia de recorrido en *entreorden*. ✓ ¡Correcto!
- ☐ b. Podrá volver a dibujar el árbol si conoce la secuencia de recorrido en *preorden* y la secuencia de recorrido en *postorden*.
- ☐ c. Podrá volver a dibujar el árbol si sólo conoce la secuencia de recorrido en *entre orden*.
- ☒ d. Podrá volver a dibujar el árbol si conoce la secuencia de recorrido en *preorden* y la secuencia de recorrido en *entreorden*. ✓ ¡Correcto!

¡Correcto!

Las respuestas correctas son:

Podrá volver a dibujar el árbol si conoce la secuencia de recorrido en *preorden* y la secuencia de recorrido en *entreorden*.Podrá volver a dibujar el árbol si conoce la secuencia de recorrido en *postorden* y la secuencia de recorrido en *entreorden*.

Pregunta 6

Correcta

Se puntúa 1 sobre 1

¿En qué condiciones el **recorrido en preorden** de un **árbol binario de búsqueda** visitará los nodos del árbol en secuencia ordenada de menor a mayor?

Seleccione una:

- ☐ a. Nunca.
- ☒ b. Sólo si el árbol es una diagonal con todos los hijos izquierdos nulos. ✓ ¡Correcto!
- ☐ c. Sólo si el árbol es una diagonal con todos los hijos derechos nulos.
- ☐ d. Siempre.

¡Correcto!

La respuesta correcta es:

Sólo si el árbol es una diagonal con todos los hijos izquierdos nulos.

Pregunta 7

Correcta

Se puntúa 1 sobre 1

¿Qué es lo peor que puede pasar con un árbol de búsqueda de n nodos si no se lo mantiene equilibrado (o balanceado) (o sea, intuitivamente, si los n nodos se distribuyen en más de $\log(n)$ niveles)?

Seleccione una:

- ☐ a. Que una búsqueda obligue a $O(2^n)$ comparaciones.
- ☐ b. Que una búsqueda obligue a $O(\log(n))$ comparaciones.
- ☒ c. Que una búsqueda obligue a $O(n)$ comparaciones. ✓ ¡Correcto!
- ☐ d. Que una búsqueda obligue a $O(n^2)$ comparaciones.

¡Correcto!

La respuesta correcta es: Que una búsqueda obligue a $O(n)$ comparaciones.

Pregunta 8

Correcta

Se puntúa 1 sobre 1

Típicamente (y por diversas razones) en un árbol binario de búsqueda no se acepta la inserción de claves o valores repetidos. Pero existen estrategias simples que pueden usarse para permitir esa repetición de claves. Una de ellas es la de relajar la regla de inserción de forma que los menores sigan yendo a la izquierda, pero ahora los **mayores o iguales** vayan a la derecha.

¿Cuáles de las siguientes afirmaciones son ciertas si en un árbol binario de búsqueda se cambia la regla de inserción para aplicar la regla relajada citada más arriba? (Más de una respuesta puede ser cierta, por lo que marque todas las que considere correctas).

Seleccione una o más de una:

- ☒ a. Dos o más claves iguales podrán quedar separadas por dos o más niveles de distancia. ✓ ¡Correcto!
- ☒ b. Si solo se insertan claves que sean iguales a la que contiene la raíz, el árbol tendrá estructura de una diagonal con todos los enlaces izquierdos nulos. ✓ ¡Correcto!
- ☐ c. Si solo se insertan claves que sean iguales a la que contiene la raíz, el árbol tendrá estructura de una diagonal con todos los enlaces derechos nulos.
- ☒ d. Dos claves iguales quedarán siempre en el árbol una como hija derecha e inmediata de la otra. ✗ Incorrecto...

¡Correcto!

Las respuestas correctas son:

Dos o más claves iguales podrán quedar separadas por dos o más niveles de distancia.,

Si solo se insertan claves que sean iguales a la que contiene la raíz, el árbol tendrá estructura de una diagonal con todos los enlaces izquierdos nulos.

Pregunta 9

Correcta

Se puntúa 1 sobre 1

¿Cuál es la razón por la que es recomendable usar una pila como soporte al recorrer un árbol binario (y por lo tanto, cuál es la razón por la que es recomendable usar recursión para ese recorrido)?

Seleccione una:

- ☐ a. No hay ninguna razón para usar una pila o para usar recursión. Un ciclo y una referencia de persecución serían siempre suficientes para hacer el recorrido completo.
- ☐ b. En realidad, es mejor usa una cola en lugar de una pila. Una cola permite almacenar en orden FIFO las direcciones de los nodos que se acaban de visitar en el recorrido, y cuando se necesite regresar a ese nodo luego de procesar alguno de sus dos hijos, su dirección quedará en el frente de la cola y será accesible en forma directa.
- ☒ c. Porque una pila permite almacenar en orden LIFO las direcciones de los nodos que se acaban de visitar en el recorrido, y cuando se necesite regresar a ese nodo luego de procesar alguno de sus dos hijos, su dirección quedará en el tope de la pila y será accesible en forma directa. ✓ ¡Correcto!
- ☐ d. No es necesario ni usar una pila ni usar recursión. El árbol puede recorrerse simplemente con un ciclo que comience en la raíz y luego se desplace vuelta tras vuelta hacia el hijo izquierdo, hasta llegar a un hijo izquierdo nulo, momento en el cual el ciclo debe detenerse y el árbol queda completamente recorrido.

¡Correcto!

La respuesta correcta es:

Porque una pila permite almacenar en orden LIFO las direcciones de los nodos que se acaban de visitar en el recorrido, y cuando se necesite regresar a ese nodo luego de procesar alguno de sus dos hijos, su dirección quedará en el tope de la pila y será accesible en forma directa.

Pregunta 10

Correcta

Se puntúa 1 sobre 1

Sabemos que para que un árbol binario de búsqueda favorezca búsquedas rápidas, el árbol debería tener cierta estructura a la que hemos designado como balance o equilibrio. No hemos dado una definición formal y detalla del concepto de balance o equilibrio (ya que eso escapa al alcance de esta asignatura) en un árbol binario: solo hemos tomado una idea intuitiva y muy conceptual.

En base a esa idea intuitiva, ¿cuál de las siguientes describe **conceptualmente** mejor lo que se entiende por un árbol balanceado o equilibrado?

Seleccione una:

- ☒ a. Intuitivamente, un árbol binario está balanceado si su altura es la mínima posible (o se acerca mucho a esa mínima) para la cantidad de nodos que tiene el árbol. ✓ ¡Correcto!
- ☐ b. Intuitivamente, un árbol binario está balanceado si tiene forma de zig-zag, con no más de un nodo en cada nivel del árbol.
- ☐ c. Intuitivamente, un árbol binario está balanceado si su altura es la máxima posible (o se acerca mucho a esa máxima) para la cantidad de nodos que tiene el árbol.
- ☐ d. Intuitivamente, un árbol binario está balanceado si tiene forma de diagonal (hacia la derecha o hacia la izquierda).

¡Correcto!

La respuesta correcta es:

Intuitivamente, un árbol binario está balanceado si su altura es la mínima posible (o se acerca mucho a esa mínima) para la cantidad de nodos que tiene el árbol.

◀ [Materiales Adicionales para la Ficha 29](#)

Ir a...

[Guía 29 de Ejercicios Prácticos ►](#)

Comenzado el	miércoles, 5 de marzo de 2025, 22:58
Estado	Finalizado
Finalizado en	miércoles, 5 de marzo de 2025, 23:10
Tiempo empleado	11 minutos 23 segundos
Puntos	12/14
Calificación	8 de 10 (83%)

Pregunta 1

Correcta

Se puntúa 1 sobre 1

¿Cuáles de las siguientes afirmaciones referidas a conceptos básicos generales sobre grafos son correctas? (Más de una puede ser correcta...)

Seleccione una o más de una:

- ☒ a. Un grafo puede tener nodos aislados: nodos que no tienen ningún arco incidente a él (ninguno de entrada y ninguno de salida). ✓ ¡Correcto!
- ☒ b. Un grafo puede tener subgrafos no unidos entre sí por arcos que los conecten (o sea, un grafo puede tener dos o más componentes conexas). ✓ ¡Correcto!
- ☐ c. Todo arco de un grafo, necesaria y obligatoriamente une dos nodos diferentes de ese grafo.
- ☒ d. Dos nodos *a* y *b* de un grafo (dirigido o no) pueden tener varios caminos de distinta longitud que los unan. ✓ ¡Correcto!

¡Correcto!

Las respuestas correctas son:

Un grafo puede tener nodos aislados: nodos que no tienen ningún arco incidente a él (ninguno de entrada y ninguno de salida),

Un grafo puede tener subgrafos no unidos entre sí por arcos que los conecten (o sea, un grafo puede tener dos o más componentes conexas),

Dos nodos *a* y *b* de un grafo (dirigido o no) pueden tener varios caminos de distinta longitud que los unan.

Pregunta 2

Correcta

Se puntúa 1 sobre 1

¿Cuál de las siguientes afirmaciones es FALSA respecto de los ciclos en un grafo?

Seleccione una:

- ☐ a. Es válido que un ciclo conste de un único arco que permita vincular a un nodo consigo mismo.
- ☐ b. Un ciclo es un camino que parte de un nodo y lleva de vuelta al mismo nodo.
- ☒ c. Los ciclos no están permitidos en un grafo. ✓ ¡Correcto! Esta afirmación es claramente falsa, que es lo que se estaba pidiendo indicar.
- ☐ d. Los ciclos pueden ser válidos o no, dependiendo del problema que se quiere resolver.

¡Correcto!

La respuesta correcta es:

Los ciclos no están permitidos en un grafo.

Pregunta 3

Correcta

Se puntúa 2 sobre 2

Suponga que se desea modelar con un grafo la estructura del plan de estudios de una carrera universitaria. El equipo de trabajo ha decidido que los vértices del grafo representen asignaturas y los arcos representen restricciones de correlativas previas. Así, el arco (x, y) que parte de la asignatura x y llega a la y indica que para cursar y debe estar regularizada x . Lo anterior implica además que el grafo debe ser dirigido. El equipo de trabajo ha determinado además algunas restricciones obvias que deben controlarse y validarse en todo momento en el sistema de gestión del plan. Indique cuáles de las siguientes situaciones **no serían aceptables** en el grafo propuesto, y deberían ser evitadas por procesos de validación (aclaración: **más de una puede ser seleccionada**):

Seleccione una o más de una:

- ☒ a. La ocurrencia de auto ciclos en cualquiera de sus vértices. ✓ ¡Correcto!
- ☒ b. La ocurrencia de algún tipo de relación transitiva explícita, es decir, que existan al mismo tiempo los arcos (x, y) , (y, z) y (x, z) . ✓ ¡Correcto!
- ☐ c. Vértices aislados, es decir, vértices a los que no llega ningún arco de entrada y de los cuales tampoco hay arcos de salida.
- ☐ d. Vértices a los que no llega ningún arco de entrada (aunque podrían tener arcos de salida).
- ☒ e. La ocurrencia de ciclos en general en el grafo. ✓ ¡Correcto!

¡Correcto!

Las respuestas correctas son:

La ocurrencia de ciclos en general en el grafo., La ocurrencia de auto ciclos en cualquiera de sus vértices.,

La ocurrencia de algún tipo de relación transitiva explícita, es decir, que existan al mismo tiempo los arcos (x, y) , (y, z) y (x, z) .

Pregunta 4

Correcta

Se puntúa 1 sobre 1

¿En cuál de las siguientes situaciones **no debería** usar un *grafo dirigido*?

Seleccione una:

- ☐ a. Para modelar un "sociograma": un gráfico que expresa en qué forma un grupo de personas se relacionan entre sí.
- ☐ b. Para modelar un plan de estudios de una carrera universitaria.
- ☐ c. Para modelar un sistema de cañerías que transportan líquidos desde una fuente hasta un destino.
- ☒ d. Para modelar un plano de rutas entre localidades o ciudades. ✓ ¡Correcto!

¡Correcto!

La respuesta correcta es:

Para modelar un plano de rutas entre localidades o ciudades.

Pregunta 5

Correcta

Se puntúa 1 sobre 1

¿En cuáles de las siguientes situaciones sería ideal un *grafo no dirigido*? (Más de una puede ser correcta...)

Seleccione una o más de una:

- ☒ a. Para modelar un plano de rutas entre localidades o ciudades. ✓ ¡Correcto!
- ☐ b. Para modelar un "sociograma": un gráfico que expresa en qué forma un grupo de personas se relacionan entre sí.
- ☒ c. Para modelar un sistema de rutas aéreas entre distintos aeropuertos del mundo. ✓ ¡Correcto!
- ☐ d. Para modelar un plan de estudios de una carrera universitaria.

¡Correcto!

Las respuestas correctas son:

Para modelar un plano de rutas entre localidades o ciudades.,

Para modelar un sistema de rutas aéreas entre distintos aeropuertos del mundo.

Pregunta 6

Parcialmente correcta

Se puntúa 1 sobre 1

¿Cuáles de las siguientes expresan afirmaciones **ciertas** cuando se quiere implementar un **grafo ponderado** en forma matricial simple? (Más de una puede ser correcta...)

Seleccione una o más de una:

- ☐ a. Si se almacena solo un entero para representar al arco y su peso, sería inviable representar arcos con pesos múltiples.
- ☒ b. Si se almacena directamente el peso del arco en cada casillero, un problema posible es el de la ambigüedad del cero: no sería posible distinguir si el cero es el peso de un arco, o el indicador de que el arco no existe. ✓ ¡Correcto!
- ☐ c. El problema no existe: no hay manera de representar grafos ponderados en forma matricial.
- ☒ d. Una manera recomendable de implementar un grafo ponderado en forma matricial, es hacer que cada casilla de matriz de adyacencias apunte a un objeto que represente a un arco, y guardar en ese objeto todos los atributos que sean necesarios para la representación del arco, ✓ ¡Correcto!

Las respuestas correctas son:

Si se almacena directamente el peso del arco en cada casillero, un problema posible es el de la ambigüedad del cero: no sería posible distinguir si el cero es el peso de un arco, o el indicador de que el arco no existe.,

Si se almacena solo un entero para representar al arco y su peso, sería inviable representar arcos con pesos múltiples.,

Una manera recomendable de implementar un grafo ponderado en forma matricial, es hacer que cada casilla de matriz de adyacencias apunte a un objeto que represente a un arco, y guardar en ese objeto todos los atributos que sean necesarios para la representación del arco,

Pregunta 7

Incorrecta

Se puntúa 0 sobre 2

Un *grafo reflexivo* es un grafo dirigido en el cual se cumple que todo vértice está relacionado consigo mismo (es decir, todo vértice tiene un auto ciclo). Suponga un grafo G dirigido y reflexivo con n vértices, implementado en forma matricial y asumiendo que la matriz de adyacencias almacena valores boolean en forma directa. ¿Qué se puede afirmar que es **verdadero** respecto de la matriz de adyacencias de G ?

Seleccione una:

- ☒ a. Todos los casilleros de la diagonal principal de la matriz de adyacencia valen *True*, y sólo ellos pueden valer *True*. ✗ Incorrecto.
- ☐ b. Todos los casilleros de la matriz de adyacencia valen *True*.
- ☐ c. Todos los casilleros del triángulo superior de la matriz de adyacencia valen *True*.
- ☐ d. Todos los casilleros de la diagonal principal de la matriz de adyacencia valen *True*, pero no es obligatorio que sólo ellos puedan valer *true*.

La respuesta correcta es:

Todos los casilleros de la diagonal principal de la matriz de adyacencia valen *True*, pero no es obligatorio que sólo ellos puedan valer *true*.

Pregunta 8

Correcta

Se puntúa 2 sobre 2

Un *grafo simétrico* es un grafo dirigido en el cual se cumple que si existe el arco (x, y) entonces existe también el arco (y, x) (es decir, toda vez que existe un arco en un sentido, existe también el arco en sentido contrario). Suponga un grafo G dirigido y simétrico con n vértices, implementado en forma matricial y asumiendo que la matriz de adyacencias almacena valores *boolean* en forma directa. ¿Qué se puede afirmar que es **verdadero** respecto de la matriz de adyacencias de G ?

Seleccione una:

- ☐ a. Todos los casilleros de la diagonal principal de la matriz de adyacencia valen *True*.
- ☒ b. La matriz es simétrica (o espejada) respecto de la diagonal principal: el valor en el casillero $ady[x][y]$ siempre será $\checkmark$ igual al valor en $ady[y][x]$. ¡Correcto!
- ☐ c. Todos los casilleros del triángulo inferior de la matriz de adyacencia valen *True* y los del triángulo superior valen *False*.
- ☐ d. Todos los casilleros de la matriz son *True* o todos los casilleros son *False*.

¡Correcto!

La respuesta correcta es:

La matriz es simétrica (o espejada) respecto de la diagonal principal: el valor en el casillero $ady[x][y]$ siempre será igual al valor en $ady[y][x]$.

Pregunta 9

Correcta

Se puntúa 2 sobre 2

Un *grafo transitivo* es un grafo dirigido en el cual se cumple que cada vez que existen los arcos (x, y) e (y, z) entonces existe también el arco (x, z) . Suponga un grafo G dirigido y transitivo con n vértices, implementado en forma matricial y asumiendo que la matriz de adyacencias almacena valores *boolean* en forma directa. ¿Qué se puede afirmar que es **verdadero** respecto de la matriz de adyacencias de G ?

Seleccione una:

- ☐ a. Si el casillero $ady[y][z]$ es *True* entonces el casillero $ady[x][z]$ será también *True*, sin importar el valor de $ady[x][y]$.
- ☒ b. Si los casilleros $ady[x][y]$ y $ady[y][z]$ son *True*, entonces el casillero $ady[x][z]$ será también *True*. $\checkmark$ ¡Correcto!
- ☐ c. Si el casillero $ady[x][y]$ es *True* entonces el casillero $ady[x][z]$ será también *True*, sin importar el valor de $ady[y][z]$.
- ☐ d. Si los casilleros $ady[x][y]$ y $ady[y][z]$ son *False*, entonces el casillero $ady[x][z]$ será *True*.

¡Correcto!

La respuesta correcta es:

Si los casilleros $ady[x][y]$ y $ady[y][z]$ son *True*, entonces el casillero $ady[x][z]$ será también *True*.

Pregunta **10**

Correcta

Se puntúa 1 sobre 1

¿Cuáles de las siguientes afirmaciones relativas a la idea arcos de entrada o de salida de un nodo son correctas? (Más de una puede ser correcta...)

Seleccione una o más de una:

- ☒ a. El Fuera de Grado de un nodo (o Grado de Salida) en un grafo dirigido es la cantidad de arcos que tienen a ese nodo como nodo de partida. ✓ ¡Correcto!
- ☒ b. El Grado de un nodo (tanto si el grafo es dirigido como si es no dirigido) es la cantidad de arcos incidentes a ese nodo (arcos que tienen al nodo como partida o como llegada). ✓ ¡Correcto!
- ☒ c. El Entre Grado de un nodo (o Grado de Entrada) en un grafo dirigido es la cantidad de arcos que tienen a ese nodo como nodo de llegada. ✓ ¡Correcto!
- ☐ d. El Grado de un nodo (tanto si el grafo es dirigido como si es no dirigido) es la cantidad de arcos no incidentes a ese nodo (arcos que no tienen a ese nodo ni como partida ni como llegada).

¡Correcto!

Las respuestas correctas son:

El Entre Grado de un nodo (o Grado de Entrada) en un grafo dirigido es la cantidad de arcos que tienen a ese nodo como nodo de llegada,

El Fuera de Grado de un nodo (o Grado de Salida) en un grafo dirigido es la cantidad de arcos que tienen a ese nodo como nodo de partida,

El Grado de un nodo (tanto si el grafo es dirigido como si es no dirigido) es la cantidad de arcos incidentes a ese nodo (arcos que tienen al nodo como partida o como llegada).

[◀ Ficha 30 \[Grafos\] \(PDF - para lectura directa\)](#)

Ir a...

[Guía 30 de Ejercicios Prácticos ▶](#)

[Página Principal](#) / [Mis cursos](#) / [AED-2024-IntLib-\(Feb2025\)](#) / [Ficha 31](#) / [Cuestionario 31 \[Temas: hasta Ficha 31\]](#)

Comenzado el	miércoles, 5 de marzo de 2025, 23:13
Estado	Finalizado
Finalizado en	miércoles, 5 de marzo de 2025, 23:27
Tiempo empleado	13 minutos 19 segundos
Puntos	12/13
Calificación	9 de 10 (92%)

Pregunta **1**

Correcta

Se puntúa 1 sobre 1

¿En qué universidad estudió, se doctoró y formó parte de su Academia de Ciencias el matemático *Wilhelm Ackerman*?

Seleccione una:

- ☐ a. Oxford
- ☐ b. Padova
- ☒ c. Göttingen ✓ ¡Correcto!
- ☐ d. Stanford

¡Correcto!

La respuesta correcta es:

Göttingen

Pregunta **2**

Correcta

Se puntúa 1 sobre 1

¿Cuál de los siguientes era el nombre verdadero del matemático conocido como [Fibonacci](#), que fue quien definió la famosa *Sucesión de Fibonacci*?



Seleccione una:

- ☐ a. Carl Gauss
- ☐ b. Giuseppe Peano
- ☒ c. Leonardo Pisano ✓ ¡Correcto!
- ☐ d. Leonhard Euler

¡Correcto!

La respuesta correcta es:

Leonardo Pisano

Pregunta 3

Incorrecta

Se puntúa 0 sobre 1

¿Cuál es la mejora esencial que el algoritmo *Quicksort* realiza sobre el algoritmo *Bubblesort* o *Burbuja*?

Seleccione una:

- ☐ a. *Quicksort* primero determina qué tan desordenado está el arreglo, mientras que *Bubblesort* procede directamente a ordenarlo
- ☐ b. *Quicksort* no implementa ninguna mejora sustancial sobre *Bubblesort*.
- ☒ c. En las versiones analizadas en clases, *Quicksort* sólo usa un **✗ Incorrecto... sólo debe revisar el código fuente de cada versión para verlo... Y en todo caso, esa no sería una mejora "esencial"...** ciclo para recorrer el arreglo, mientras que *Bubblesort* usa dos.
- ☐ d. ***Quicksort* acelera el cambio de posición tanto de los elementos menores como de los mayores, mientras que *Bubblesort* solo acelera a los mayores (o a los menores, dependiendo de la forma de implementación).**

La respuesta correcta es:

Quicksort acelera el cambio de posición tanto de los elementos menores como de los mayores, mientras que *Bubblesort* solo acelera a los mayores (o a los menores, dependiendo de la forma de implementación).

Pregunta 4

Correcta

Se puntúa 1 sobre 1

¿Cuál de los siguientes esquemas simples de pseudocódigo representa a un algoritmo planteado mediante estrategia Divide y Vencerás, pero con tiempo de ejecución $O(n \log(n))$?

Seleccione una:

- ☐ a. `proceso(partición) :`
 `adicional() [==> O(1)]`
 `proceso(partición/2)`
 `proceso(partición/2)`
- ☐ b. `proceso(partición) :`
 `adicional() [==> O(n³)]`
 `proceso(partición/2)`
 `proceso(partición/2)`
- ☐ c. `proceso(partición) :`
 `adicional() [==> O(n²)]`
 `proceso(partición/2)`
 `proceso(partición/2)`
- ☒ d. `proceso(partición) :` ✓ ¡Correcto!
 `adicional() [==> O(n)]`
 `proceso(partición/2)`
 `proceso(partición/2)`

¡Correcto!

La respuesta correcta es:

```
proceso(partición) :  
    adicional() [==> O(n)]  
    proceso(partición/2)  
    proceso(partición/2)
```

Pregunta 5

Correcta

Se puntúa 1 sobre 1

¿Cuál de las siguientes situaciones haría que el algoritmo *Quicksort* degenera en su peor caso en cuanto al tiempo de ejecución, de orden n^2 ?

Seleccione una:

- ☒ a. Que el arreglo de entrada tenga sus elementos dispuestos de tal forma que cada vez que se seleccione el pivot en cada partición, resulte que ese pivot sea siempre el menor o el mayor de la partición que se está procesando. ✓ ¡Correcto!
- ☐ b. El algoritmo Quicksort no tiene un peor caso $O(n^2)$. Su tiempo de ejecución siempre es $O(n \log(n))$.
- ☐ c. Que el arreglo esté ya ordenado, en la misma secuencia en que se lo quiere ordenar.
- ☐ d. Que el arreglo esté ya ordenado, pero al revés.

¡Correcto!

La respuesta correcta es:

Que el arreglo de entrada tenga sus elementos dispuestos de tal forma que cada vez que se seleccione el pivot en cada partición, resulte que ese pivot sea siempre el menor o el mayor de la partición que se está procesando.

Pregunta 6

Correcta

Se puntúa 1 sobre 1

¿Cuál de las siguientes estrategias de obtención del pivot *es la más recomendable* para evitar que el algoritmo *Quicksort* se degrade en su peor caso $O(n^2)$ en cuanto a su tiempo de ejecución?

Seleccione una:

- ☐ a. En cada partición, usar como pivot al valor de la última casilla.
- ☐ b. En cada partición, usar como pivot al valor de la primera casilla.
- ☐ c. En cada partición, usar como pivot al valor de la casilla central.
- ☒ d. En cada partición, obtener el pivot por la mediana de tres (ya sea la mediana entre el primero, el último y el central; o bien la mediana entre tres elementos aleatorios la partición). ✓ ¡Correcto!

¡Correcto!

La respuesta correcta es:

En cada partición, obtener el pivot por la mediana de tres (ya sea la mediana entre el primero, el último y el central; o bien la mediana entre tres elementos aleatorios la partición).

Pregunta 7

Correcta

Se puntúa 1 sobre 1

¿Por qué es considerada una *mala idea* tomar como pivot al *primer elemento* (o al *último*) de cada partición al implementar el algoritmo *Quicksort*?

Seleccione una:

- ☒ a. Porque de esa forma aumenta el riesgo de caer en el peor caso, o aproximarse al peor caso, si el arreglo estuviese ya ordenado o casi ordenado. ✓ ¡Correcto!
- ☐ b. Porque de esa forma aumenta el riesgo de caer en el peor caso, o aproximarse al peor caso, si el arreglo estuviese completamente desordenado.
- ☐ c. No es cierto que sea una mala idea. Ambas alternativas son tan buenas como cualquier otra.
- ☐ d. Porque de esa forma aumenta el riesgo de caer en el peor caso, o aproximarse al peor caso, si el tamaño n del arreglo fuese muy grande.

¡Correcto!

La respuesta correcta es:

Porque de esa forma aumenta el riesgo de caer en el peor caso, o aproximarse al peor caso, si el arreglo estuviese ya ordenado o casi ordenado.

Pregunta 8

Correcta

Se puntúa 2 sobre 2

Considere la función presentada en clases para calcular mediana entre el primero, el central y el último elemento de una partición del arreglo v delimitada por los elementos en las posiciones izq y der :

```
def get_pivot_m3(v, izq, der):  
    # calculo del pivot: mediana de tres...  
    central = int((izq + der) / 2)  
  
    if v[der] < v[izq]:  
        v[der], v[izq] = v[izq], v[der]  
  
    if v[central] < v[izq]:  
        v[central], v[izq] = v[izq], v[central]  
  
    if v[central] > v[der]:  
        v[central], v[der] = v[der], v[central]  
  
    return v[central]
```

El tamaño de la partición analizada es entonces $n = der - izq + 1$ elementos. ¿Cuál es el tiempo de ejecución de esta función, expresado en notación O y de acuerdo a ese valor de n ?

Seleccione una:

- ☐ a. $O(n^2)$
- ☐ b. $O(\log(n))$
- ☒ c. $O(1)$ ✓ ¡Correcto! Efectivamente, el cálculo no depende del tamaño de la partición, por lo que resulta de tiempo constante.
- ☐ d. $O(n)$

¡Correcto!

La respuesta correcta es:

 $O(1)$

Pregunta 9

Correcta

Se puntúa 2 sobre 2

¿Cuál es la cantidad de niveles del *árbol de invocaciones recursivas* que se genera al ejecutar el *Quicksort* para ordenar un arreglo de n elementos, en el **caso promedio**? (Es decir: ¿Cuál es la altura de ese árbol en el **caso promedio**?)

Seleccione una:

- ☐ a. $Altura = n * \log(n)$
- ☒ b. $Altura = \log(n)$ ✓ ¡Correcto!
- ☐ c. $Altura = n$
- ☐ d. $Altura = n^2$

¡Correcto!

La respuesta correcta es:

 $Altura = \log(n)$

Pregunta **10**

Correcta

Se puntúa 2 sobre 2

¿Cuál es la cantidad de niveles del *árbol de invocaciones recursivas* que se genera al ejecutar el *Quicksort* para ordenar un arreglo de n elementos, en el **peor caso**? (Es decir: ¿Cuál es la *altura* de ese árbol en ese peor caso?)

Seleccione una:

- ☒ a. **Altura = n** ✓ ¡Correcto!
- ☐ b. Altura = n^2
- ☐ c. Altura = $n * \log(n)$
- ☐ d. Altura = $\log(n)$

¡Correcto!

La respuesta correcta es:

Altura = n [◀ Ficha 31 \[Estrategias de Resolución de Problemas: Divide y Vencerás\] \(PDF - para lectura directa\)](#)

Ir a...

[Guía 31 de Ejercicios Prácticos ►](#)

[Página Principal](#) / [Mis cursos](#) / [AED-2024-IntLib-\(Feb2025\)](#) / [Ficha 32](#) / [Cuestionario 32 \[Temas: hasta Ficha 32\]](#)

Comenzado el	miércoles, 5 de marzo de 2025, 23:28
Estado	Finalizado
Finalizado en	miércoles, 5 de marzo de 2025, 23:39
Tiempo empleado	10 minutos 14 segundos
Puntos	13/14
Calificación	10 de 10 (95%)

Pregunta 1

Parcialmente correcta

Se puntúa 1 sobre 1

¿Cuáles de las siguientes afirmaciones con correctas en relación a conceptos elementales del análisis de algoritmos? (Más de una respuesta puede ser válida, por lo que marque todas las que considere correctas).

Seleccione una o más de una:

- ☐ a. El análisis del *caso promedio* es aquel en el cual se estudia el comportamiento de un algoritmo cuando debe procesar una configuración de datos que llegan en forma aleatoria.
- ☒ b. Los dos factores de eficiencia más comúnmente utilizados en el análisis de algoritmos son el tiempo de ejecución ✓ ¡Correcto!
de un algoritmo y el espacio de memoria que un algoritmo emplea.
- ☒ c. La notación Big O se usa para expresar el rendimiento de un algoritmo en terminos de ✗ Incorrecto... la notación Big O
una función que imponga una cota inferior para ese algoritmo en cuanto al factor implica una *cota superior*, y no
medido (tiempo o espacio de memoria). una cota inferior...
- ☒ d. El análisis del *peor caso* es aquel en el cual se estudia el comportamiento de un algoritmo cuando debe procesar ✓ ¡Correcto!
la configuración más desfavorable posible de los datos que recibe.

Las respuestas correctas son:

Los dos factores de eficiencia más comúnmente utilizados en el análisis de algoritmos son el tiempo de ejecución de un algoritmo y el espacio de memoria que un algoritmo emplea.,

El análisis del *peor caso* es aquel en el cual se estudia el comportamiento de un algoritmo cuando debe procesar la configuración más desfavorable posible de los datos que recibe.,

El análisis del *caso promedio* es aquel en el cual se estudia el comportamiento de un algoritmo cuando debe procesar una configuración de datos que llegan en forma aleatoria.

Pregunta 2

Correcta

Se puntúa 1 sobre 1

Dado un arreglo de n componentes... ¿qué significa decir que en el peor caso la cantidad de comparaciones que realiza el algoritmo de *búsqueda secuencial* es $O(n)$ (o sea: del *orden de n*)?

Seleccione una:

- ☒ a. Significa que en el peor caso el algoritmo hará n comparaciones. ✓ ¡Correcto!
- ☐ b. Significa que en el peor caso el algoritmo no hará ninguna comparación.
- ☐ c. Significa que en el peor caso el algoritmo siempre hará menos de n comparaciones.
- ☐ d. Significa que en el peor caso el algoritmo hará siempre más de n comparaciones.

¡Correcto!

La respuesta correcta es:

Significa que en el peor caso el algoritmo hará n comparaciones.

Pregunta 3

Correcta

Se puntúa 1 sobre 1

Para cada uno de los algoritmos básicos y/o técnicas de procesamiento generales que se indican en la columna de la izquierda, seleccione la expresión en notación *Big O* que mejor expresa el tiempo de ejecución de ese algoritmo en el peor caso:

Multiplicación de matrices cuadradas de tamaño $n \times n$.

$O(n^3)$ (n al cubo) ✓

Búsqueda secuencial en un arreglo (ordenado o desordenado).

$O(n)$ ✓

Ordenamiento por selección directa.

$O(n^2)$ (n al cuadrado) ✓

Ordenamiento rápido (Quicksort) (Considere aquí el tiempo para el caso promedio).

$O(n \log(n))$ ✓

Acceso directo a un casillero de un vector.

$O(1)$ ✓

Búsqueda binaria en un arreglo ya ordenado.

$O(\log(n))$ ✓

¡Correcto!

La respuesta correcta es: Multiplicación de matrices cuadradas de tamaño $n \times n$. → $O(n^3)$ (n al cubo), Búsqueda secuencial en un arreglo (ordenado o desordenado). → $O(n)$, Ordenamiento por selección directa. → $O(n^2)$ (n al cuadrado), Ordenamiento rápido (Quicksort) (Considere aquí el tiempo para el caso promedio). → $O(n \log(n))$, Acceso directo a un casillero de un vector. → $O(1)$, Búsqueda binaria en un arreglo ya ordenado. → $O(\log(n))$

Pregunta 4

Parcialmente correcta

Se puntúa 1 sobre 1

¿Cuáles de las siguientes afirmaciones **son ciertas** en referencia a las *Estrategias de Resolución de Problemas* que se citan? (Más de una respuesta puede ser cierta, por lo que marque todas las que considere correctas...)

Seleccione una o más de una:

- ☒ a. La estrategia de *Fuerza Bruta* se basa en aplicar ideas intuitivas y directas, simples de codificar, pero normalmente produce algoritmos de mal rendimiento en tiempo de ejecución y/o de espacio de memoria empleado. ✓ ¡Correcto!
- ☐ b. La estrategia de *Backtracking* es de base recursiva y permite implementar soluciones de prueba y error explorando las distintas soluciones y volviendo atrás si se detecta que un camino conduce a una solución incorrecta. cuando es aplicable, es más eficiente que la Fuerza Bruta, ya que permite eliminar caminos por deducción.
- ☒ c. La técnica de Programación Dinámica se basa en calcular los resultados de los subproblemas de menor orden o tamaño que pudieran aparecer, almacenar esos resultados en una tabla, y luego re-usarlos cuando vuelvan a ser requeridos en el cálculo del problema mayor. ✓ ¡Correcto!
- ☐ d. El empleo de la *Recursividad* para resolver un problema no es recomendable en ningún caso, debido a la gran cantidad de recursos de memoria o de tiempo de ejecución que implica.

Las respuestas correctas son:

La estrategia de *Fuerza Bruta* se basa en aplicar ideas intuitivas y directas, simples de codificar, pero normalmente produce algoritmos de mal rendimiento en tiempo de ejecución y/o de espacio de memoria empleado.,

La estrategia de *Backtracking* es de base recursiva y permite implementar soluciones de prueba y error explorando las distintas soluciones y volviendo atrás si se detecta que un camino conduce a una solución incorrecta. cuando es aplicable, es más eficiente que la Fuerza Bruta, ya que permite eliminar caminos por deducción.,

La técnica de Programación Dinámica se basa en calcular los resultados de los subproblemas de menor orden o tamaño que pudieran aparecer, almacenar esos resultados en una tabla, y luego re-usarlos cuando vuelvan a ser requeridos en el cálculo del problema mayor.

Pregunta 5

Correcta

Se puntúa 1 sobre 1

¿Cuál de las siguientes afirmaciones es **falsa** en relación a las *Estrategias de Resolución de Problemas* (o *Estrategias de Planteo de Algoritmos*)?

Seleccione una:

- ☐ a. Se trata de un conjunto de técnicas diversas que podrían ayudar a encontrar la solución a un problema, pero sin garantía de éxito, y aún si se llega a una solución, tampoco se garantiza que esa solución sea eficiente.
- ☐ b. Se trata de un conjunto de técnicas diversas que podrían ayudar a encontrar la solución a un problema, pero sin garantía de éxito.
- ☒ c. Son técnicas y recomendaciones para el planteo de problemas que garantizan que se encontrará una solución, empleando la estrategia correcta para cada problema que se enfrente. ✓ ¡Correcto! Justamente, esta afirmación es falsa... ninguna técnica garantiza el éxito en el campo de la resolución de problemas...
- ☐ d. Un mismo problema podría ser resuelto en base a dos o más estrategias de resolución diferentes, dando lugar a distintos algoritmos para ese mismo problema.

¡Correcto!

La respuesta correcta es:

Son técnicas y recomendaciones para el planteo de problemas que garantizan que se encontrará una solución, empleando la estrategia correcta para cada problema que se enfrente.

Pregunta 6

Correcta

Se puntúa 1 sobre 1

Para problema general nombrado en la columna de la izquierda, seleccione la estrategia de planteo de algoritmos que se sabe haya resultado más útil para resolver ese problema, o bien la que sea que haya podido aplicarse para resolverlo aún sin llegar a una solución eficiente (considere a cada problema en su situación más general, y no casos particulares de cada uno):

Problema de las Ocho Reinas	Backtracking	✓
Generación de gráficos fractales.	Recursión	✓
Problema del árbol de expansión mínimo de un grafo.	Algoritmo ávido	✓
Problema de la alineación de secuencias.	Programación dinámica	✓
Ordenamiento rápido (Quicksort).	Divide y vencerás	✓
Problema del Viajante.	Fuerza Bruta $[O(n!)]$ / Programación Dinámica $[O(n^2 * 2^n)]$	✓

¡Correcto!

La respuesta correcta es: Problema de las Ocho Reinas → Backtracking, Generación de gráficos fractales. → Recursión, Problema del árbol de expansión mínimo de un grafo. → Algoritmo ávido, Problema de la alineación de secuencias. → Programación dinámica, Ordenamiento rápido (Quicksort). → Divide y vencerás, Problema del Viajante. → Fuerza Bruta $[O(n!)]$ / Programación Dinámica $[O(n^2 * 2^n)]$

Pregunta 7

Correcta

Se puntúa 1 sobre 1

¿Cuáles de los siguientes son conocidos algoritmos basados en la estrategia *Divide y Vencerás*? (Más de una respuesta puede ser correcta, por lo que marque todas las que considere válidas)

Seleccione una o más de una:

- ☒ a. Algoritmo *Mergesort* para ordenamiento de arreglos. ✓ ¡Correcto!
- ☐ b. Algoritmo *Shellsort* para ordenamiento de arreglos.
- ☐ c. Algoritmo *Heapsort* para ordenamiento de arreglos.
- ☒ d. Algoritmo *Quicksort* para ordenamiento de arreglos. ✓ ¡Correcto!

¡Correcto!

Las respuestas correctas son:

Algoritmo *Quicksort* para ordenamiento de arreglos.,

Algoritmo *Mergesort* para ordenamiento de arreglos.

Pregunta 8

Correcta

Se puntúa 1 sobre 1

¿Cuál de las siguientes resume en forma correcta la idea general de la estrategia *Divide y Vencerás* para resolución de problemas?

Seleccione una:

- ☐ a. Se usa una tabla para almacenar los resultados de los subproblemas que se hayan calculado, y luego cuando algún subproblema vuelve a aparecer se toma su valor desde la tabla, para evitar pérdida de tiempo.
- ☒ b. El conjunto de n datos se divide en subconjuntos de aproximadamente el mismo tamaño ($n/2$, $n/3$, $n/4$, etc.). Luego se aplica recursión para procesar cada uno de esos subconjuntos. Finalmente se unen las partes que se acaban de procesar para lograr el resultado final. ✓ ¡Correcto!
- ☐ c. Se aplica una regla simple que parece ser beneficiosa, sin volver atrás ni medir las consecuencias de aplicar esa regla, con la esperanza de lograr el resultado óptimo al final.
- ☐ d. El conjunto de n datos se divide en subconjuntos de cualquier tamaño, sin importar si los tamaños de cada subconjunto coinciden entre sí. Luego se aplica recursión para procesar cada uno de esos subconjuntos. Finalmente se unen las partes que se acaban de procesar para lograr el resultado final.

¡Correcto!

La respuesta correcta es:

El conjunto de n datos se divide en subconjuntos de aproximadamente el mismo tamaño ($n/2$, $n/3$, $n/4$, etc.). Luego se aplica recursión para procesar cada uno de esos subconjuntos. Finalmente se unen las partes que se acaban de procesar para lograr el resultado final.

Pregunta 9

Correcta

Se puntúa 1 sobre 1

Considere el problema del Cambio de Monedas analizado en clases, y la solución mediante un Algoritmo Ávido también presentada en clases ¿Cuáles de las siguientes afirmaciones **son ciertas** en relación al problema y al algoritmo citado? (Más de una respuesta puede ser cierta, por lo que marque todas las que considere correctas...)

Seleccione una o más de una:

- ☐ a. Si el Problema de Cambio de Monedas no puede resolverse en forma óptima para un conjunto dado de monedas que incluya a la de 1 centavo, mediante el Algoritmo Ávido propuesto, entonces el problema no tiene solución.
- ☐ b. El Algoritmo Ávido sugerido para el Problema del Cambio de Monedas falla si el valor x a cambiar tiene una moneda igual a x en el conjunto de valores nominales: en ese caso, el algoritmo provoca un error de runtime y se interrumpe.
- ☐ c. El Algoritmo Ávido sugerido para el problema del Cambio de Monedas funciona correctamente para cualquier conjunto de valores nominales de monedas, siempre y cuando ese conjunto incluya a la moneda de 1 centavo.
- ☒ d. Sea cual sea el algoritmo que se emplee, es exigible que exista la moneda de 1 centavo, pues de otro modo no habrá solución posible para muchos valores de cambio. ✓ ¡Correcto!

¡Correcto!

La respuesta correcta es:

Sea cual sea el algoritmo que se emplee, es exigible que exista la moneda de 1 centavo, pues de otro modo no habrá solución posible para muchos valores de cambio.

Pregunta **10**

Correcta

Se puntúa 1 sobre 1

Considere el problema del *Cambio de Monedas* analizado en clases, y la solución mediante *Programación Dinámica* también presentada en clases ¿Cuáles de las siguientes afirmaciones **son ciertas** en relación al problema y al algoritmo citado? (Más de una respuesta puede ser cierta, por lo que marque todas las que considere correctas...)

Seleccione una o más de una:

- ☒ a. En el algoritmo basado en Programación Dinámica los valores de las monedas que sean mayores a x , son dejados de lado y la recurrencia de cálculo no se aplica sobre ellos. ✓ ¡Correcto!
- ☐ b. En el algoritmo basado en Programación Dinámica, el resultado final a retornar es igual a la suma o acumulación de todos los valores almacenados en el arreglo *prev* donde se almacenaron los resultados intermedios.
- ☒ c. En el algoritmo basado en Programación Dinámica no es importante si el arreglo *coins* está ordenado o desordenado: funcionará correctamente de todas formas. ✓ ¡Correcto!
- ☒ d. En el algoritmo basado en Programación Dinámica, el resultado final a retornar es el que haya quedado almacenado en la casilla x del arreglo *prev* que contiene los resultados intermedios (o sea, en *prev[x]*). ✓ ¡Correcto!

¡Correcto!

Las respuestas correctas son:

En el algoritmo basado en Programación Dinámica no es importante si el arreglo *coins* está ordenado o desordenado: funcionará correctamente de todas formas,

En el algoritmo basado en Programación Dinámica los valores de las monedas que sean mayores a x , son dejados de lado y la recurrencia de cálculo no se aplica sobre ellos.,

En el algoritmo basado en Programación Dinámica, el resultado final a retornar es el que haya quedado almacenado en la casilla x del arreglo *prev* que contiene los resultados intermedios (o sea, en *prev[x]*).

Pregunta **11**

Correcta

Se puntúa 1 sobre 1

Considere el problema de las *Ocho Reinas* presentado en clases. ¿Cuáles de las siguientes afirmaciones son **ciertas** en relación a las **diagonales del tablero** en el cual deben colocarse la reinas, suponiendo que el tablero es el normal del ajedrez, de 8×8 ? (Más de una respuesta puede ser cierta, por lo que marque todas las que considere correctas...)

Seleccione una o más de una:

- ☒ a. En cada una de las diagonales que se orientan como la contra-diagonal o diagonal inversa, es constante la suma entre el número de columna y el número de fila de cada uno de sus elementos. ✓ ¡Correcto!
- ☒ b. La cantidad **total** de diagonales que contiene el tablero (sumando todas las diagonales de todos los tipos posibles) es 30. ✓ ¡Correcto! Efectivamente... son 15 normales y 15 inversas...
- ☒ c. En cada una de las diagonales que se orientan como la principal, es constante la resta entre el número de columna y el número de fila de cada uno de sus elementos. ✓ ¡Correcto!
- ☒ d. En general hay dos tipos de diagonales: las normales (orientadas en la misma forma que la diagonal principal) y las inversas (orientadas en la misma forma que la contra-diagonal o diagonal inversa). ✓ ¡Correcto!

¡Correcto!

Las respuestas correctas son:

En general hay dos tipos de diagonales: las normales (orientadas en la misma forma que la diagonal principal) y las inversas (orientadas en la misma forma que la contra-diagonal o diagonal inversa),

La cantidad **total** de diagonales que contiene el tablero (sumando todas las diagonales de todos los tipos posibles) es 30,

En cada una de las diagonales que se orientan como la principal, es constante la resta entre el número de columna y el número de fila de cada uno de sus elementos.,

En cada una de las diagonales que se orientan como la contra-diagonal o diagonal inversa, es constante la suma entre el número de columna y el número de fila de cada uno de sus elementos.

Pregunta 12

Correcta

Se puntúa 1 sobre 1

Considere el problema de las *Ocho Reinas* presentado en clases. ¿Cuáles de las siguientes afirmaciones son **ciertas** en relación a las **diagonales normales del tablero** en el cual deben colocarse la reinas, suponiendo que el tablero es el normal del ajedrez, de 8×8 ? (Más de una respuesta puede ser cierta, por lo que marque todas las que considere correctas...)

Seleccione una o más de una:

- ☐ a. El valor de la suma entre el número de columna y el número de fila de cada componente de una diagonal normal, es un número constante para cada diagonal, y los posibles valores están en el intervalo $[0..14]$
- ☒ b. Las diagonales normales pueden representarse con un arreglo *qnd* de 15 componentes, en el que cada diagonal cuyos elementos tengan el mismo valor $(col - fil)$, se haga coincidir el casillero *qnd* $[(col - fil) + 7]$ (evitando de esta forma los índices negativos). ✓ ¡Correcto!
- ☒ c. El valor de la resta entre el número de columna y el número de fila de cada componente de una diagonal normal, es un número constante para cada diagonal, y los posibles valores están en el intervalo $[-7..7]$. ✓ ¡Correcto!
- ☐ d. Las diagonales normales pueden representarse con un arreglo *qnd* de 15 componentes, en el que cada diagonal cuyos elementos tengan el mismo valor $(col + fil)$, se haga coincidir el casillero *qnd* $[(col + fil)]$.

¡Correcto!

Las respuestas correctas son:

El valor de la resta entre el número de columna y el número de fila de cada componente de una diagonal normal, es un número constante para cada diagonal, y los posibles valores están en el intervalo $[-7..7]$,

Las diagonales normales pueden representarse con un arreglo *qnd* de 15 componentes, en el que cada diagonal cuyos elementos tengan el mismo valor $(col - fil)$, se haga coincidir el casillero *qnd* $[(col - fil) + 7]$ (evitando de esta forma los índices negativos).

Pregunta 13

Correcta

Se puntúa 1 sobre 1

Considere el problema de las *Ocho Reinas* presentado en clases. ¿Cuáles de las siguientes afirmaciones son **ciertas** en relación a las **diagonales inversas del tablero** en el cual deben colocarse la reinas, suponiendo que el tablero es el normal del ajedrez, de 8×8 ? (Más de una respuesta puede ser cierta, por lo que marque todas las que considere correctas...)

Seleccione una o más de una:

- ☒ a. El valor de la suma entre el número de columna y el número de fila de cada componente de una diagonal inversa, es un número constante para cada diagonal, y los posibles valores están en el intervalo $[0..14]$. ✓ ¡Correcto!
- ☒ b. Las diagonales inversas pueden representarse con un arreglo *qid* de 15 componentes, en el que cada diagonal cuyos elementos tengan el mismo valor $(col + fil)$, se haga coincidir el casillero *qid* $[(col + fil)]$. ✓ ¡Correcto!
- ☐ c. El valor de la resta entre el número de columna y el número de fila de cada componente de una diagonal inversa, es un número constante para cada diagonal, y los posibles valores están en el intervalo $[-7..7]$
- ☐ d. Las diagonales inversas pueden representarse con un arreglo *qid* de 15 componentes, en el que cada diagonal cuyos elementos tengan el mismo valor $(col - fil)$, se haga coincidir el casillero *qid* $[(col - fil) + 7]$.

¡Correcto!

Las respuestas correctas son:

El valor de la suma entre el número de columna y el número de fila de cada componente de una diagonal inversa, es un número constante para cada diagonal, y los posibles valores están en el intervalo $[0..14]$,

Las diagonales inversas pueden representarse con un arreglo *qid* de 15 componentes, en el que cada diagonal cuyos elementos tengan el mismo valor $(col + fil)$, se haga coincidir el casillero *qid* $[(col + fil)]$.

Pregunta **14**

Correcta

Se puntúa 1 sobre 1

Considere el problema de las *Ocho Reinas* presentado en clases. Se ha indicado que se puede usar un arreglo rc de componentes, en el cual el casillero $rc[col] = fil$ indica que la reina de la columna col está ubicada en la fila fil . ¿Cuáles de las siguientes configuraciones para el arreglo rc representan **soluciones incorrectas** para el problema de las *Ocho Reinas*? (Más de una respuesta puede ser cierta, por lo que marque todas las que considere correctas...)

Seleccione una o más de una:

- ☒ a. $rc = [2, 0, 7, 4, 5, 1, 6, 3]$ ✓ ¡Correcto! Efectivamente, si rc contiene los valores mostrados, **hay al menos** un problema: las reinas de las columnas 3 y 4 están ubicadas en la misma diagonal normal: la diagonal -1.
- ☐ b. $rc = [4, 7, 3, 0, 2, 5, 1, 6]$
- ☐ c. $rc = [5, 3, 6, 0, 7, 1, 4, 2]$
- ☒ d. $rc = [3, 5, 7, 0, 5, 1, 2, 4]$ ✓ ¡Correcto! Efectivamente, si rc contiene los valores mostrados, hay al menos un problema: las reinas de las columnas 1 y 4 están ubicadas en la misma fila: la 5.

¡Correcto!

Las respuestas correctas son:

 $rc = [3, 5, 7, 0, 5, 1, 2, 4]$, $rc = [2, 0, 7, 4, 5, 1, 6, 3]$ [◀ Materiales Adicionales para la Ficha 32](#)

Ir a...

[Guía 32 de Ejercicios Prácticos ▶](#)